



User Interface Design (23CSE113)

LAB MANUAL

Submitted By:	
Name	MAHESWARAN R
Roll No	AV.SC.U4CSE25127
Year/Sem/Section	1st YEAR /SEM-2/CSE-B
Date of Submission	
Submitted To:	
Name	Dr. Raj Kumar Batchu
Department	CSE
Designation	Assistant Professor, School of Computing

INDEX

SNO	LAB PROGRAM	PAGENO	REMARKS
1.	1(A)To understand the installation process of a VS code and get familiar with its environment for web development. 1(B)To learn the basic structure of an HTML document and create a simple webpage using headings and paragraphs. 1(C)To understand and implement basic HTML tags such as image, hyperlink, text formatting, and inline styling to design a simple webpage.	4-6 7-8 8-10	
2.	2(A) To design a simple web page using HTML that demonstrates the use of background color, background image, heading styling, and preformatted text. 2(B)To create a static webpage using only HTML that displays basic information about Amrita Vishwa Vidyapeetham, including a heading, descriptive text, background styling through HTML attributes, and a table showing the list of campuses in different states. 2(C) To design a restaurant's menu webpage using HTML and displaying menu items in a table format with proper layout and visual styling.	11-12 12-18 18-22	
3.	3(A)To design and develop an academic timetable using HTML tables by applying row span, column span, and styling for proper structure and presentation. 3(B) To create a comprehensive webpage that includes an academic timetable along with syllabus representation using images and structured HTML elements.	23-28 28-39	
4.	4(A)To understand the concept of forms and design a structured student registration form using HTML tables, including various input types such as text fields, radio buttons, checkboxes, file upload, and dropdown lists. 4(B) To understand the concept of forms and design user login and registration webpages using HTML, and to display them within a single interface using iframe for better layout and user interaction.	40-43 44-47	

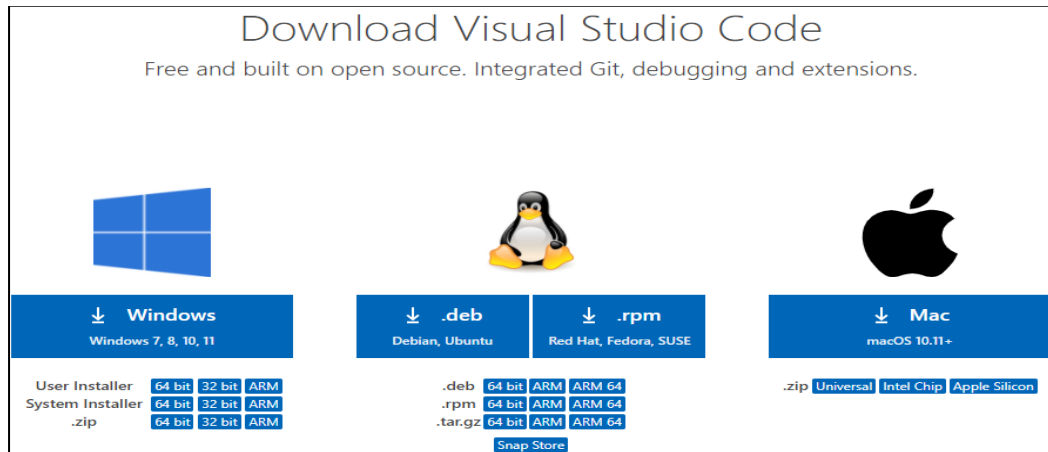
5.	5(A) To create a responsive card layout using HTML and CSS with three cards, each containing an image, title, and button.	48-53	
	5(B) To design a mini website using iframe with navigation links (Home, About, Contact), where clicking on each link displays the respective content within the same page using internal or external CSS.	54-60	
	5(C) To design an Amrita login page using HTML and CSS that displays different campuses, where clicking on each campus shows its details on the same page.	61-71	
6.	Design a responsive e-commerce application using the grid layout. This application should contain different categories of products such as electronics, clothing, home-appliances, books. Based on the category we select corresponding products should be displayed on the sidebar.	72-77	
7.	To create the repository in GIT Hub and push or upload files of all the programs that are done to create different webpages and websites.	78-79	
8.	8a) create a webpage that displays a square that continuously rotates 360 using css animation.	80-81	
	8b) create a ball that moves up and down continuously using css animations.	81-82	
	8c) create a circle that move horizontally and rotates at the same time.	82-84	
9.	Create an webpage using html, CSS, java script based on the given conditions: a) write a JS program to determine if a number is even or odd b) write a JS program to determine if a number is prime or not c) write a JS program to find the factorial of a number d) write a JS program to find the given number is Armstrong number or not		
10.	a) Write a JS conditional statement to find the sign of a product of three numbers using functions. Display an alert box with the specified sign. b) Write a JS conditional statement to sort three numbers using functions. Display an alert box to show the result. c) Write a JS conditional statement to find the largest of five numbers using function & onclick event. Display an alert box to show the result. d) Write a JS for loop that will iterate from 0 to 15. For each iteration, it will check if the current number is odd or even & display a message to the screen. e) Write a JS program which iterates the integers from 1 to 100.		

	But for multiples of three print "LOL" instead of the number and for multiples of five print "OK". For numbers which are multiples of both three & five print "LOL-OK".		
11.	a) A website requires users to enter a username. The system should display a message while typing if the username is less than 5 characters. b) A registration form should validate email format before submission. If invalid, the form should not be submitted. c) A system should check password strength while typing and display whether it is weak or strong. d) A form should validate a mobile number when the user leaves the input field. The number must be exactly 10 digits. e) Design an HTML page, such that the registration form must validate: Name (not empty) Email (valid format) Password (minimum 6 characters)		
12.	A) To understand the installation process of a electron.		

LAB-1

1(A) To understand the installation process of a VS code and get familiar with its environment for web development:

[Step 1]: Visit the Official Website of the Visual Studio Code using any web browser like [Google Chrome](#), [Microsoft Edge](#), etc.



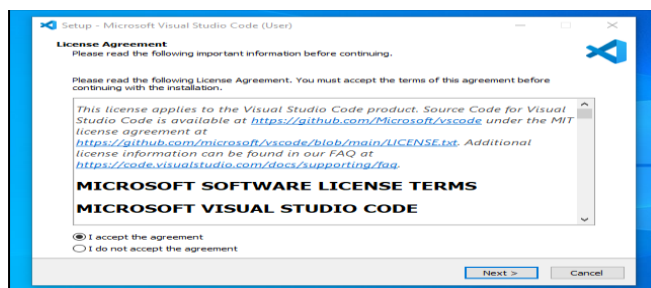
[Step 2]: Press the "Download for Windows" button on the website to start the download of the Visual Studio Code Application.

[Step 3]: When the download finishes, then the Visual Studio Code Icon appears in the downloads folder.

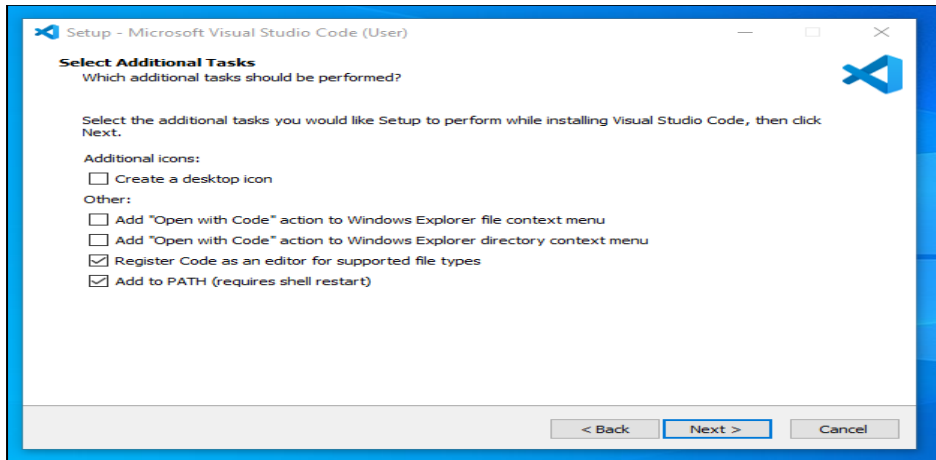


[Step 4]: Click on the Installer icon to start the installation process of the Visual Studio Code.

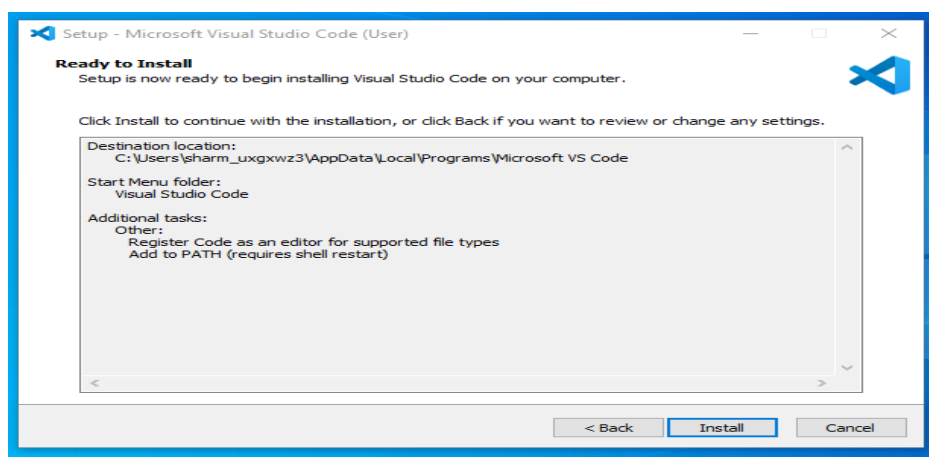
[Step 5]: After the Installer opens, it will ask you to accept the terms and conditions of the Visual Studio Code. Click on I accept the agreement and then click the Next button.



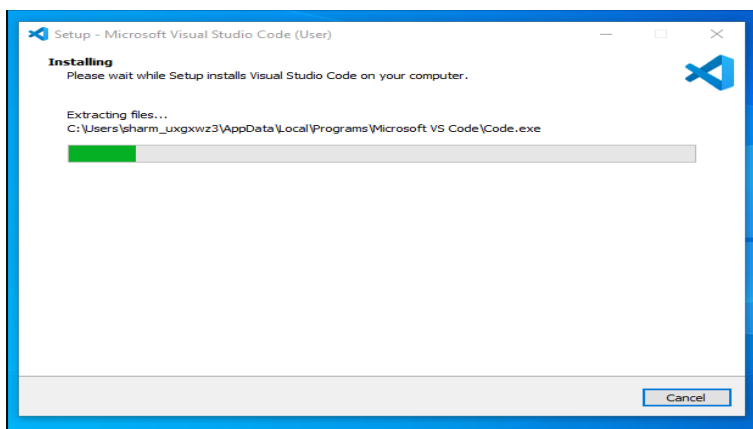
[Step 6]: Choose the location data for running the Visual Studio Code. It will then ask you to browse the location. Then click on the Next button.



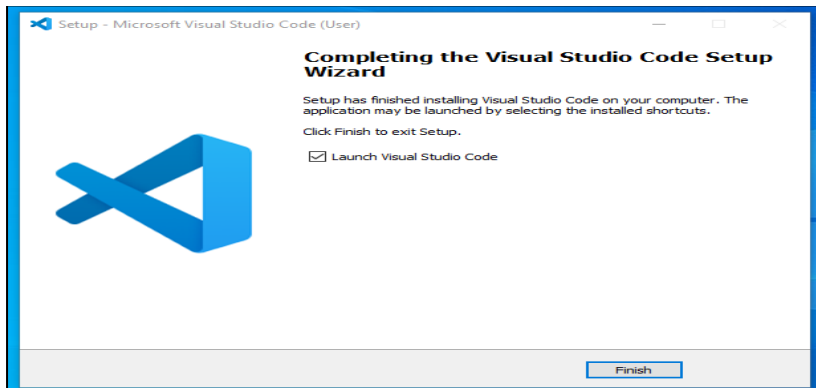
[Step 7]: Then it will ask to begin the installation setup. Click on the Install button.



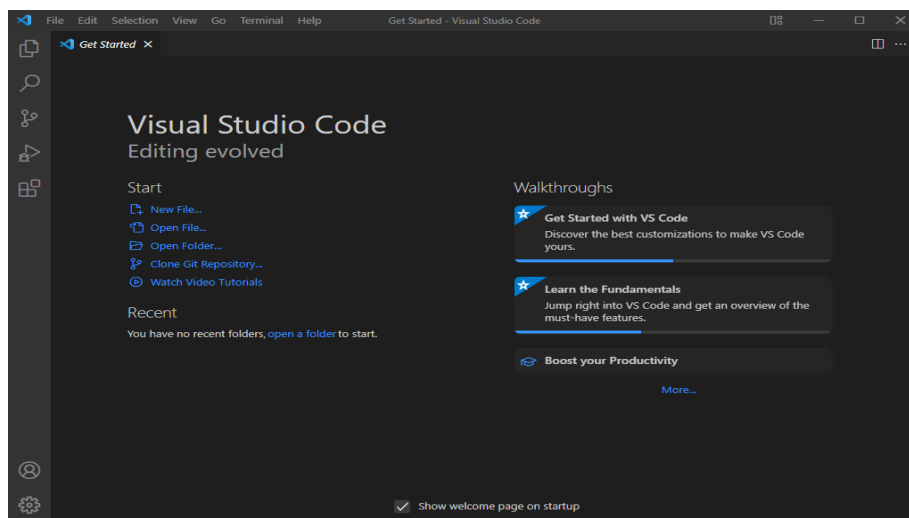
[Step 8]: After clicking on Install, it will take about 1 minute to install the Visual Studio Code on your device.



[Step 9]: After the Installation setup for Visual Studio Code is finished, it will show a window like this below. Tick the "Launch Visual Studio Code" checkbox and then click Next.



[Step 10]: After the previous step, the Visual Studio Code window opens successfully. Now you can create a new file in the Visual Studio Code window and choose a language to begin programming.



1(B) To learn the basic structure of an HTML document and create a simple webpage using headings and paragraphs:

A) **Aim:** To learn the basics of HTML

Program:

```
<!DOCTYPE html>

<html>

  <head>

    <title>

      My First Webpage

    </title>

  </head>

  <body>

    <p style="background-color: aliceblue; text-align: center; border: 1px; border-color: black; border-style: dashed;">Name: MAHESWARAN R Roll no: AV.SC.U4CSE25127 Sec: CSE-B</p>

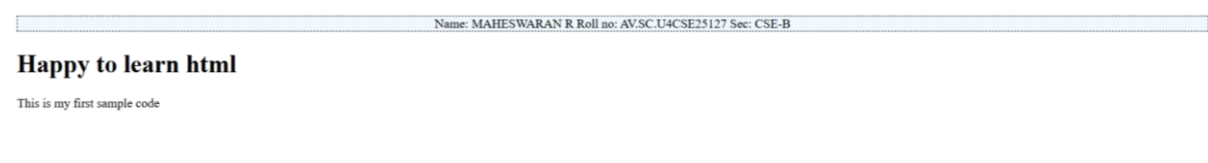
    <h1>Happy to learn html</h1>

    <p>This is my first sample code</p>

  </body>

</html>
```

Output:



Explanation of the Tags:

- HTML stands for Hyper Text Markup Language.
- HTML is used to create and structure webpages.
- It tells the web browser what to display and how to display it.
- **<!DOCTYPE html>:** This tag tells the browser that the above document is an HTML 5 document.
- **<html>:** This tag is the root element of every HTML page.

- **<head>**: This tag contains meta information about the HTML page.
- **<title>**: This tag specifies a title for the HTML page.
- **<body>**: This tag contains all visible content like text, images, links, tables etc.
- **<h1>**: The <h1> tag defines a large heading.
- **<p>**: The <p> element defines a paragraph.

1(C) To understand and implement basic HTML tags such as image, hyperlink, text formatting, and inline styling to design a simple webpage:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
</head>
```

```
<body>
```

```
  <title>heading examples</title>
```

```
</head>
```

```
<body>
```

```
  <h2>Paragraph</h2>
```

```
  <p>Name:maheswaran</p>
```

```
  <p>Roll.no:CSE25127.</p>
```

```
  <p>Class:CSE-B.</p>
```

```
  <h2>Headings</h2>
```

```
  <h1>This is heading 1</h1>
```

```
  <h2>This is heading 2</h2>
```

```
  <h3>this is heading 3</h3>
```

```
  <h4>This is heading 4</h4>
```

```
  <h5>This is heading 5</h5>
```

<h6>This is heading 6</h6>

<h2>Attributes</h2>

<h2>HTML links</h2>

Amrita

Vit

Srm

</body>

</body>

</html>

OUTPUT: -

Paragraph

Name:maheswaran

Roll.no:CSE25127.

Class:CSE-B.

Headings

This is heading 1

This is heading 2

this is heading 3

This is heading 4

This is heading 5

This is heading 6

Attributes



HTML links

[Amrita](https://www.amrita.edu)

[Vit](https://vit.ac.in)

[Srm](https://www.srmist.edu.in)

Explanation of tags:

This HTML code creates a simple webpage displaying personal details using `<p>` tags and different levels of headings using `<h1>` to `<h6>`. It also includes images using the `<img>` tag with attributes like `src`, `width`, and `height`, and hyperlinks using `<a>` to navigate to websites.

LAB-2

2(A) To design a simple web page using HTML that demonstrates the use of

background color, background image, heading styling, and preformatted text:

Program:

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>styles example</title>

<style>

.highlight {
background-color: rgba(146, 2, 67, 0.149);
font-weight: bold;
}

.italic
font-style: italic;
}

</style>

</head>

<body>

<p class="highlight">This paragraph is highlighted with a red background and bold text.</p>

<p class="italic">This paragraph is styled in italics.</p>

</body>

</html>
```

OUTPUT:

This paragraph is highlighted with a red background and bold text.

This paragraph is styled in italics.

Explanation:

This HTML code creates a webpage that demonstrates simple CSS styling using classes. The `<style>` section inside the `<head>` defines two classes: `.highlight` adds a light red background and makes the text bold, while `.italic` is intended to make text italic.

In the `<body>`, two paragraphs are displayed using `<p>` tags, each applying one of the CSS classes with the class attribute. The first paragraph appears highlighted and bold, and the second should appear in italics.

However, there is a syntax error in the `.italic` class (missing `}`), which prevents the italic style from being applied correctly.

2(B) To create a static webpage using only HTML that displays basic information about Amrita Vishwa Vidyapeetham, including a heading, descriptive text, background styling through HTML attributes, and a table showing the list of campuses in different states.

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1">

  <title>Amrita Vishwavidyapeetham</title>

  <style>

    body {

      margin: 0;

      padding: 0;

      font-family: "Times New Roman", serif;

      background-image: url('amrita1.png'); /* Replace with your image */

      background-size: cover;

      background-position: center;

      background-repeat: no-repeat;

      text-align: center;

      color: #002147;

      background-color: #f0f8ff; /* fallback if image missing */

    }

    .header {

      border: 3px solid violet;

      padding: 10px;

      font-size: 40px;

      font-weight: bold;
```

```
    color: brown;

    background-color: rgba(255,255,255,0.6);
}

.description {
    width: 70%;
    margin: auto;
    font-size: 18px;
    line-height: 1.8;
    font-style: italic;
    padding: 20px;
}

.branch-section {
    width: 75%;
    margin: auto;
    background-color: rgba(255,255,255,0.6);
    padding: 10px;
}

.branch-title {
    background-color: #5cd6c6;
    font-size: 24px;
    font-weight: bold;
    padding: 5px;
}

table {
    width: 100%;
    margin-top: 10px;
    border-collapse: separate;
    border-spacing: 10px;
}
```

```
th {  
    background-color: #f2e6c9;  
    padding: 8px;  
    border: 1px solid black;  
    font-weight: bold;  
}  
td {  
    padding: 8px;  
    border: 1px solid black;  
}  
.sno {  
    background-color: #1abc9c;  
    color: black;  
    font-weight: bold;  
}  
.campus {  
    background-color: #16a085;  
    color: black;  
}  
.location {  
    background-color: #d4e157;  
}  
.contact-btn {  
    padding: 5px 10px;  
    margin: 2px;  
    border: none;  
    cursor: pointer;  
}  
.grey-btn {
```

```

        background-color: #cccccc;
    }

    .purple-btn {
        background-color: purple;
        color: white;
    }

    .caption-style {
        caption-side: top;
        font-weight: bold;
        padding: 6px;
    }

    .student-info {
        background-color: aliceblue;
        text-align: center;
        border: 1px dashed black;
    }
</style>
</head>
<body>
    <header class="header">
        <h1>Amrita Vishwavidyapeetham</h1>
    </header>
    <p class="student-info">Name: MAHESWARAN Roll no: AV.SC.U4CSE25127
    Sec: CSE-B</p>
    <main>
    <div class="description">
        Mata Amritanandamayi Devi (born Sudhamani Idamannel on September 27, 1953),
        known fondly as Amma ("Mother") or the "hugging saint", is a world-renowned
        Indian spiritual leader and humanitarian from Kerala. She is globally recognized

```

for embracing millions as a profound act of compassion, offering unconditional love to alleviate physical and emotional suffering.

</div>

<div class="branch-section">

<div class="branch-title">

AMRITA VISHWAVIDYAPEETHAM BRANCHES

</div>

<table aria-label="Amrita campuses list">

<caption class="caption-style">Campus Contact List</caption>

<tr>

<th scope="col">S.No</th>

<th scope="col">Campus</th>

<th scope="col">Location</th>

<th scope="col">Contact No</th>

<th scope="col">Contact Person</th>

</tr>

<tr>

<td class="sno">1</td>

<td class="campus">Amaravati Campus</td>

<td class="location">Kuragallu</td>

<td>+91-9865784585</td>

<td>

Rajkumar

Batchu

</td>

</tr>

<tr>

<td class="sno">2</td>

<td class="campus">Amritapuri Campus</td>

```

        <td class="location">Amrita Nagar</td>
        <td>+91-9231184586</td>
        <td>
            <a href="tel:+919231184586">Adithya Ram</a>
        </td>
    </tr>
    <tr>
        <td class="sno">3</td>
        <td class="campus">Coimbatore Campus</td>
        <td class="location">Ettimadai</td>
        <td>+91-9123456789</td>
        <td>
            <a href="tel:+919123456789" >Aashish</a>
            <a href="tel:+919123456780" >B</a>
        </td>
    </tr>
</table>
</div>
</main>
</body>
</html>

```

OUTPUT:



Explanation:

This HTML code creates a simple webpage to demonstrate CSS styling using classes. The `<style>` section inside the `<head>` defines two classes: `.highlight` for adding a light red background and bold text, and `.italic` for making text italic.

In the `<body>`, two paragraphs are created using `<p>` tags, each applying a different class through the class attribute. The first paragraph uses the `.highlight` class, so it appears with a colored background and bold text.

The second paragraph is intended to use the `.italic` class to display italic text. However, there is a syntax error in the CSS (missing `{` in `.italic`), which prevents the style from being applied correctly.

2(C) To design a restaurant's menu webpage using HTML and displaying menu items in a table format with proper layout and visual styling.

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Document</title>

</head>

<body>

    <style>

        body {

            background-image: url('hotel3.png');

            background-size: cover;

            background-position: center;

            background-repeat: repeat-x;

            height: 100vh;

            margin: 0;

        }

    </style>

    <h1 align="center">Hotel Menu</h1>

    <pre align="center">

        <table border="1" cellspacing="0" cellpadding="10">

            <style>

                table {

                    border-collapse: collapse;

                    width: 50%;

                    margin: auto;
```

```

    }
    th, td {
        border: 1px solid #000;
        padding: 10px;
        font-size: 15px;
        text-align: center;
        color: #e8e3ec;
    }
    th {
        font-size: 25px;
        color: #b97858;
    }
}
</style>

<p class="student-info">Name: MAHESWARAN Roll no: AV.SC.U4CSE25127
Sec: CSE-B</p>

<tr>
    <th>Item</th>
    <th>Price (INR)</th>
</tr>
<tr>
    <td>Veg Biryani</td>
    <td>150</td>
</tr>
<tr>
    <td>Chicken Biryani</td>
    <td>200</td>
</tr>
<tr>
    <td>Paneer Butter Masala</td>
    <td>120</td>

```

```

</tr>

<tr>

    <td>Butter Chicken</td>

    <td>180</td>

</tr>

<tr>

    <td>Veg Noodles</td>

    <td>90</td>

</tr>

<tr>

    <td>Chicken Noodles</td>

    <td>120</td>

</tr>

<tr>

    <td>Veg Pizza</td>

    <td>150</td>

</tr>

<tr>

    <td>Chicken Pizza</td>

    <td>180</td>

</tr>

<tr>

    <td>Veg Burger</td>

    <td>60</td>

</tr>

<tr>

    <td>Chicken Burger</td>

    <td>80</td>

</tr>

```

</body>

</html>

OUTPUT:



Name: MAHESWARAN Roll no: AV.SC.UACSE25127 Sec: CSE-B	
Item	Price (INR)
Veg Biryani	150
Chicken Biryani	200
Paneer Butter Masala	120
Butter Chicken	180
Veg Noodles	90
Chicken Noodles	120
Veg Pizza	150
Chicken Pizza	180
Veg Burger	60
Chicken Burger	80

Explanation:

This HTML code creates a simple hotel menu webpage with a background image and a centered table of food items. The <style> inside the <body> sets the page background using an image and controls its size, position, and repetition.

A main heading <h1> displays “Hotel Menu” at the center of the page. The <table> is used to list food items and their prices, with rows (<tr>) and columns (<th>, <td>).

Additional CSS inside the table styles its layout, including borders, spacing, text alignment, and colors. The menu includes items like biryani, noodles, pizza, and burgers with their prices.

However, there are some structural issues such as using <style> and <p> tags inside the table and <pre> tag unnecessarily. Overall, the code demonstrates how to design a basic styled menu using HTML and CSS.

LAB-3

3(A) To design and develop an academic timetable using HTML tables by applying row span, column span, and styling for proper structure and presentation.

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <title>Master Time Table – CSE II Semester</title>

  <style>

    body {

      font-family: Arial, sans-serif;

      background-image: url("mountain.png");

      background-repeat: no-repeat;

      background-size: cover;

      background-position: center;

      background-attachment: fixed;

    }

    h2, h3 {

      text-align: center;

      margin: 5px;

      color: #000;

    }

    .info-table, .timetable, .course-table {

      width: 95%;

      margin: 15px auto;

      border-collapse: collapse;

      background: rgba(255, 255, 255, 0.9);

    }
```

```

th, td {
    border: 1px solid black;
    padding: 8px;
    text-align: center;
    font-size: 14px;
}

th {
    background-color: #e6e6e6;
}

.break {
    background-color: yellow;
    font-weight: bold;
    writing-mode: vertical-rl;
    transform: rotate(180deg);
}

.counselling {
    color: red;
    font-weight: bold;
}

.left {
    text-align: left;
}

</style>
</head>
<body>

    <p class="student-info">Name: MAHESWARAN Roll no: AV.SC.U4CSE25127
    Sec: CSE-B</p>

    <h2>Master Time Table – CSE II Semester (B Section)</h2>

    <h3>Department of Computer Science & Engineering</h3>

```

<table class="info-table">

<tr>

<td class="left">Class: B.Tech CSE-B</td>

<td class="left">Semester: II</td>

<td class="left">Class Room: 511</td>

</tr>

<tr>

<td colspan="3" class="left">

Class Advisors:

1. Dr. Kurmala Gowri Raghavendra Narayan

Email: kgr_narayan@av.amrita.edu | Mobile: +91 9705409393

2. Dr. Ankit Sachan

Email: s_ankit@av.amrita.edu | Mobile: +91 8381800129

</td>

</tr>

</table>

<table class="timetable">

<tr>

<th>Day</th>

<th>Slot 1
08:55–09:45</th>

<th>Slot 2
09:45–10:35</th>

<th rowspan="6" class="break">Short Break</th>

<th>Slot 3
10:45–11:35</th>

<th>Slot 4
11:35–12:25</th>

<th>Slot 5
12:25–01:15</th>

<th rowspan="6" class="break">Lunch Break</th>

<th>Slot 6
02:10–03:00</th>

<th rowspan="6" class="break">Short Break</th>

<th>Slot 7 03:10–04:00</th>
<th>Slot 8 04:00–04:50</th>
</tr>
<tr>
<th>Monday</th>
<td colspan="2">23MAT117 Linear Algebra LAB (BYOD)</td>
<td>LIBRARY</td>
<td>23PHY115 Modern Physics</td>
<td>LIBRARY</td>
<td>23MEE115 Manufacturing Practice LAB</td>
<td colspan="2">23MEE115 Manufacturing Practice LAB</td>
</tr>
<tr>
<th>Tuesday</th>
<td>23MAT116 Discrete Mathematics</td>
<td>23MAT117 Linear Algebra</td>
<td>23PHY115 Modern Physics</td>
<td>23CSE111 OOP</td>
<td>23CSE113 UID</td>
<td>Library / Sports</td>
<td colspan="2">22ADM111 Glimpses of Glorious India</td>
</tr>
<tr>
<th>Wednesday</th>
<td colspan="2">23CSE111 OOP Lab</td>
<td>23MAT116 Discrete Mathematics</td>
<td>23CSE113 UID</td>
<td>22ADM111 Glimpses of Glorious India</td>

```

        <td class="counselling">COUNSELLING</td>
        <td colspan="2">Library / Sports</td>
    </tr>
    <tr>
        <th>Thursday</th>
        <td>23MAT117<br>Linear Algebra</td>
        <td>23CSE111<br>OOP</td>
        <td>23MAT116<br>Discrete Mathematics</td>
        <td class="counselling">COUNSELLING</td>
        <td>LIBRARY</td>
        <td>23PHY115<br>Modern Physics (T)</td>
        <td colspan="2">Library / Sports</td>
    </tr>
    <tr>
        <th>Friday</th>
        <td colspan="2">23CSE113<br>UID Lab (BYOD)</td>
        <td>23MAT117<br>Linear Algebra</td>
        <td>23CSE111<br>OOP</td>
        <td>Library / Sports</td>
        <td>22ADM111<br>Glimpses of Glorious India (P)</td>
        <td colspan="2">23MAT116<br>Discrete Math Lab</td>
    </tr>
</table>
</body>
</html>

```

OUTPUT: -

Name: MAHESWARAN Roll no: AV.SC.U4CSE25127 Sec: CSE-B

Master Time Table – CSE II Semester (B Section)
Department of Computer Science & Engineering

Class: B.Tech CSE-B	Semester: II	Class Room: 511
----------------------------	---------------------	------------------------

Class Advisors:
 1. Dr. Kumala Gowri Raghavendra Narayan
 Email: kgr_narayan@av.amrita.edu | Mobile: +91 9705409393
 2. Dr. Ankit Sachan
 Email: s_ankit@av.amrita.edu | Mobile: +91 8381800129

18 19 20 21 22

Time/Day	slot 1 8:55am-9:45am	Slot 2 9:45am-10:35am	10:35am-10:45am	Slot 3 10:45am-11:35am	Slot 4 11:35am-12:25pm	Slot 5 12:25pm-1:15pm	1:15pm-2:05pm	Slot 6 2:10pm-3:00pm	3:00pm-3:10pm	Slot 7 3:10pm-4:00pm	Slot 8 4:00pm-4:50pm
Monday	Linear Algebra LAB		Short break	LIBRARY	Modern Physics	LIBRARY	Lunch Break	Manufacturing Practice LAB	Short break	Manufacturing Practice LAB	
Tuesday	Discrete Mathematics	Linear Algebra		Modern Physics	Object Oriented Programming	User Interface Design		LIBRARY		Glimpses of Glorious India	
Wednesday	Object Oriented Programming			Discrete Mathematics	User Interface Design	Glimpses of Glorious India		COUNSELLING		LIBRARY	
Thursday	Linear Algebra	Object Oriented Programming		Discrete Mathematics	COUNSELLING	LIBRARY		Modern Physics		LIBRARY	
Friday	User Interface Design LAB BYOD			Linear Algebra	Object Oriented Programming	LIBRARY		Glimpses of Glorious India		Discrete Mathematics LAB	

Course Code	Course Name	L-T-P	Credits	Faculty Name	Assisting Faculty
23MAT116	Discrete Mathematics	3-0-2	4	Dr.Srivivasa Rao Thota	
23MAT117	Linear Algebra	3-0-2	4	Mr.Sapavat Bixapathi	
23CSE111	Object Oriented Programming	3-0-2	4	Dr. Balaji TK	
23PHY115	Modern Physics	2-1-0	3	Dr. C R Kesavalu	
23CSE113	User Interface Design	2-0-2	3	Dr.Rajkumar Batchu	Mr. Barge Bharadwaj
23MEE115	Manufacturing Practice	0-0-3	1	Dr. Sathies S.	
22ADM111	Glimpses of Glorious India	2-0-1	2	Dr. Siva Panuganti	
Total (15L + 1Tut +12P =28 hrs)		15-1-12	21		

Explanation:

This HTML code creates a styled timetable webpage for a CSE II Semester class. The `<style>` section defines the layout, including fonts, a full-page background image, and table styling for better readability.

Headings `<h2>` and `<h3>` are used to display the title and department name at the center. The first table (info-table) shows class details like semester, classroom, and faculty advisors.

The main timetable is created using another `<table>` with rows (`<tr>`) and columns (`<th>`, `<td>`), organizing subjects across days and time slots. Special classes like `.break` are used to highlight breaks vertically, and `.counselling` highlights counselling sessions in red.

CSS properties like `colspan` and `rowspan` are used to merge cells for labs and breaks. Overall, the code demonstrates how to design a structured and visually styled timetable using HTML and CSS.

3(B) To create a comprehensive webpage that includes an academic timetable along with syllabus representation using images and structured HTML elements.

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Semester 2 Syllabus</title>

<style>

  body {

    background-image: url('mountain.png');

    background-repeat: no-repeat;

    background-size: cover;

    background-position: center;

    font-family: Arial, sans-serif;

    background-color: beige;

  }

  h1 {

    text-align: center;

    margin: 20px;

    color: maroon;

  }

  .syllabus-grid {

    display: grid;

    grid-template-columns: repeat(2, 1fr);

    gap: 20px;

    width: 90%;

    margin: 0 auto;
```

```

    }

    .subject-card {
        background-color: white;
        border: 2px solid maroon;
        border-radius: 10px;
        padding: 10px;
        text-align: center;
    }

    .subject-card img {
        width: 100%;
        height: auto;
        border-bottom: 1px solid maroon;
    }

    .subject-name {
        margin-top: 10px;
        font-size: 18px;
        font-weight: bold;
        color: navy;
    }
}
</style>
</head>
<body>
<p class="student-info">Name: MAHESWARAN Roll no: AV.SC.U4CSE25127 Sec: CSE-
B</p>
<div class="container">
    <h1>Semester 2 Syllabus</h1>

    <div class="syllabus-grid">
        <div class="subject-card">

```

```

    
    <div class="subject-name">Discrete Mathematics</div>
</div>

<div class="subject-card">
    
    <div class="subject-name">Linear Algebra</div>
</div>

<div class="subject-card">
    
    <div class="subject-name">Modern Physics</div>
</div>

<div class="subject-card">
    
    <div class="subject-name">manufacturing practices</div>
</div>

<div class="subject-card">
    
    <div class="subject-name">Object Oriented Programming</div>
</div>

<div class="subject-card">
    
    <div class="subject-name">User Interface Design</div>
</div>
</div>
</div>
</body>
<h2>Master Time Table – CSE II Semester</h2>
<style>

```

```

body {
    background-image: url('mountain.png');
    background-repeat: no-repeat;
    background-size: cover;
    background-position: center;
    font-family: Arial, sans-serif;
    background-color: beige;
}

h2, h3 {
    text-align: center;
    margin: 5px;
}

.info-table, .timetable, .course-table {
    width: 95%;
    margin: 15px auto;
    border-collapse: collapse;
    background: white;
}

th, td {
    border: 1px solid black;
    padding: 8px;
    text-align: center;
    font-size: 14px;
}

th {
    background-color: #e6e6e6;
}

.break {
    background-color: yellow;
}

```

```

    font-weight: bold;
    writing-mode: vertical-rl;
    transform: rotate(180deg);
}

.counselling {
    color: red;
    font-weight: bold;
}

.left {
    text-align: left;
}

</style>
</head>
<body>
<h2>Master Time Table – CSE II Semester (B Section)</h2>
<h3>Department of Computer Science & Engineering</h3>
<table class="info-table">
    <tr>
        <td class="left"><b>Class:</b> B.Tech CSE-B</td>
        <td class="left"><b>Semester:</b> II</td>
        <td class="left"><b>Class Room:</b> 511</td>
    </tr>
    <tr>
        <td colspan="3" class="left">
            <b>Class Advisors:</b><br>
            1. Dr. Kurmala Gowri Raghavendra Narayan<br>
            Email: kgr_narayan@av.amrita.edu | Mobile: +91 9705409393<br></td>
        <td>
            <br>
        </td>
    </tr>

```

2. Dr. Ankit Sachan

Email: s_ankit@av.amrita.edu | Mobile: +91 8381800129

</td>

</tr>

</table>

<table class="timetable">

<tr>

<th>Time/Day</th>

<th>slot1
08:55–09:45</th>

<th>slot1
09:45–10:35</th>

<th rowspan="6" class="break">Short Break</th>

<th>slot3
10:45–11:35</th>

<th>slot4
11:35–12:25</th>

<th>slot5
12:25–01:15</th>

<th rowspan="6" class="break">Lunch Break</th>

<th>slot6
02:10–03:00</th>

<th rowspan="6" class="break">Short Break</th>

<th>slot7
03:10–04:00</th>

<th>slot8
04:00–04:50</th>

</tr>

<tr>

<th>Monday</th>

<td colspan="2">23MAT117
Linear Algebra LAB (BYOD)</td>

<td>LIBRARY</td>

<td>23PHY115
Modern Physics</td>

<td>LIBRARY</td>

<td>23MEE115
Manufacturing Practice LAB</td>

<td colspan="2">23MEE115
Manufacturing Practice LAB</td>

</tr>

```

<tr>

<th>Tuesday</th>

<td>23MAT116<br>Discrete Mathematics</td>

<td>23MAT117<br>Linear Algebra</td>

<td>23PHY115<br>Modern Physics</td>

<td>23CSE111<br>Object Oriented Programming</td>

<td>23CSE113<br>User Interface Design</td>

<td>Library / Sports</td>

<td colspan="2">22ADM111<br>Glimpses of Glorious India</td>

</tr>

<tr>

<th>Wednesday</th>

<td colspan="2">23CSE111<br>OOP Lab</td>

<td>23MAT116<br>Discrete Mathematics</td>

<td>23CSE113<br>User Interface Design</td>

<td>22ADM111<br>Glimpses of Glorious India</td>

<td class="counselling">COUNSELLING</td>

<td colspan="2">Library / Sports</td>

</tr>

<tr>

<th>Thursday</th>

<td>23MAT117<br>Linear Algebra</td>

<td>23CSE111<br>OOP</td>

<td>23MAT116<br>Discrete Mathematics</td>

<td class="counselling">COUNSELLING</td>

<td>LIBRARY</td>

<td>23PHY115<br>Modern Physics (T)</td>

<td colspan="2">Library / Sports</td>

</tr>

```

```

<tr>
  <th>Friday</th>
  <td colspan="2">23CSE113<br>UID Lab (BYOD)</td>
  <td>23MAT117<br>Linear Algebra</td>
  <td>23CSE111<br>OOP</td>
  <td>Library / Sports</td>
  <td>22ADM111<br>Glimpses of Glorious India (P)</td>
  <td colspan="2">23MAT116<br>Discrete Mathematics Lab</td>
</tr>

```

```

</table>

```

```

<table class="course-table">

```

```

  <tr>
    <th>Course Code</th>
    <th>Course Name</th>
    <th>L-T-P</th>
    <th>Credits</th>
    <th>Faculty Name</th>
    <th>Assisting Faculty</th>
  </tr>
  <tr>
    <td>23MAT116</td>
    <td>Discrete Mathematics</td>
    <td>3-0-2</td>
    <td>4</td>
    <td>Dr. Srivivasa Rao Thota</td>
    <td>-</td>
  </tr>
  <tr>
    <td>23MAT117</td>

```

<td>Linear Algebra</td>
<td>3-0-2</td>
<td>4</td>
<td>Mr. Sapavat Bixapathi</td>
<td>-</td>

<td>23CSE111</td>
<td>Object Oriented Programming</td>
<td>3-0-2</td>
<td>4</td>
<td>Dr. Balaji TK</td>
<td>-</td>

<td>23PHY115</td>
<td>Modern Physics</td>
<td>2-1-0</td>
<td>3</td>
<td>Dr. C R Kesavalu</td>
<td>-</td>

<td>23CSE113</td>
<td>User Interface Design</td>
<td>2-0-2</td>
<td>3</td>
<td>Dr. Rajkumar Batchu</td>
<td>Mr. Barge Bharadwaj</td>

```

</tr>

<tr>

<td>23MEE115</td>

<td>Manufacturing Practice</td>

<td>0-0-3</td>

<td>1</td>

<td>Dr. Sathies S</td>

<td>-</td>

</tr>

<tr>

<td>22ADM111</td>

<td>Glimpses of Glorious India</td>

<td>2-0-1</td>

<td>2</td>

<td>Dr. Siva Panuganti</td>

<td>-</td>

</tr>

<tr>

<th colspan="3">Total</th>

<th>21</th>

<th colspan="2">(15L + 1T + 12P = 28 Hrs)</th>

</tr>

</table>

</body>

</html>

```

Output: -

Name: MAHESWARAN Roll no: AV.SC.U4CSE25127 Sec: CSE-B

Semester 2 Syllabus

Syllabus

Unit I

Logic, Mathematical Reasoning and Counting: Logic, Propositional Equivalence, Predicate and Quantifiers, Theorem Proving, Recursive Definitions, Recursive Algorithms, Basics of Counting, Pigeonhole Principle, Permutation and Combinations.

Unit II

Relations and Their Properties: Representing Relations, Closure of Relations, Partial Ordering, Equivalence Relations and partitions.

Advanced Counting Techniques and Relations: Recurrence Relations, Solving Recurrence Relations, Generating Functions, Solutions of Homogeneous Recurrence Relations, Divide and Conquer Relations, Inclusion Exclusion.

Unit III

Graph Theory: Graphs and Sub-graphs, isomorphism, matrices associated with graphs, degrees, walks, connected graphs, shortest path algorithms, Euler and Hamilton Graphs, Euler graphs, Euler's theorem, Fleury's algorithm for Eulerian trails, Hamilton cycles, Chinese postman problem, approximate solutions of travelling salesman problem, Closest neighbour algorithm.

Lab Practice Problems

Verifications of logical statements, truth table, satisfiability, recursive algorithms, Graph problems, degree, shortest path algorithms, Euler's algorithm and closest neighbour algorithm.

Discrete Mathematics

Syllabus

Unit I

Vector Spaces: Vector spaces - Sub-spaces - Linear Independence - Basis - Dimension; Inner Product Spaces; Inner products - Orthogonality - Orthogonal basis - Gram Schmidt Process - Change of basis - Orthogonal complements - Projection on subspace - Least Square Principle - QR- Decomposition.

Unit II

Functions of several variables: Functions, limit and continuity, Partial differentiations, total derivatives, differentiation of implicit functions and transformation of coordinates by Jacobian, Taylor's series for two variables, Vector Differentiation: Vector and Scalar Functions, Derivatives, Curves, Tangents, Arc Length, Curves in Mechanics, Velocity and Acceleration, Gradient of a Scalar Field, Directional Derivatives, Divergence of a Vector Field, Curl of a Vector Field.

Unit III

Eigen values and Eigen vectors: Eigen Values and Eigen Vectors, Diagonalization, Orthogonal Diagonalization, Quadratic Forms, Diagonalizing Quadratic Forms, Conic Sections. Similarity of linear transformations - Diagonalization and its applications - Jordan form and rational canonical form, Case Studies: Applications on least square and image transformations.

Linear Algebra

Syllabus

Unit I

Origin of quantum theory of radiation: Black body radiation, photo-electric effect, Compton Effect - pair production and annihilation, De-Broglie hypothesis, description of waves and wave packets, group velocities, Evidence for wave nature of particles: Davisson-Germer experiment, Heisenberg uncertainty principle.

Unit II

Atomic structure: Historical Development of atomic structures: Thomson's Model, Rutherford's Model, Scattering formula and its predictions, Atomic spectra - Bohr's Model, Sommerfeld's Model, The

Home About Campus Academics Research Q

Unit III

Quantum mechanics: Wave function, Probability density, expectation values - Schrodinger equation - time dependent and independent, Linearity and superposition, expectation values, operators, Eigen functions and Eigen values

Unit IV

Application of 1D Schrodinger Wave equation: Free particle, Particle in a box, Finite potential well, Tunnel effect, Harmonic oscillator.

Unit V

Intro to Quantum computing-Q bit-II Quantum correlations: Bell inequalities and entanglement, Schmidt decomposition, super dense coding, teleportation, Module

Modern Physics

Syllabus

Syllabus

1. Additive Manufacturing Laboratory -12 hours

Introduction to digital manufacturing, Introduction to Additive Manufacturing - types - additive manufacturing applications - Materials for 3D printing, CAD Modeling for Additive manufacturing, Slicing and STL file generation - G-code generation - 3D printing of simple geometries.

2. Mechanical Engineering Laboratory -18 hours

Study of tools and equipment used for basic manufacturing processes. Manual arc welding practice for making Butt and Lap joints - Soldering Practice Introduction to Machine Tools and Machining Processes

3. Automation lab -12 hours

Design, simulate and test pneumatic and electro-pneumatic circuits. Introduction to PLC-PLC programming for automation applications.

4. Automobile Engineering lab -9 hours

Overview of automobiles - components -functioning of various sub-systems: Power train, steering system, suspension system and braking system. Introduction to electric vehicles, hybrid vehicles, alternate fuels, Introduction to EV Mobility.

Unit II

Introduction to Raspberry Pi, GPIO programming, interfacing sensors and actuators. Introduction to IoT, communicating sensor data to cloud platforms.

manufacturing practices

Syllabus

Unit I

Structural to Object Oriented Approach by Examples - Object Oriented languages - Properties of Object Oriented system - UML and Object Oriented Software Development - Use case diagrams and documents as a functional model - Identifying Objects and classes - Representation of Objects and its state by Object Diagram - Simple Class using class diagram - Encapsulation - Data hiding - Reading and Writing Objects - Class Level and Instance Level Attributes and Methods - JUnit environment for debugging

Unit II

Aggregation and Composition using Class Diagram - Generalization using Class Diagram - Inheritance - Constructor and Over Riding - Visibility - Attribute - Parameter - Package - Local and Global - Polymorphism - Overloading - Abstract Classes and Interfaces.

Unit III

Exception Handling - Inner Classes - Wrapper classes - String - and String Builder classes - Number - Math - Random - Array methods - File Streams - Serialization - Generics - Collection Framework - Comparator and Comparable - Vector and ArrayList - Iterator and Iterable.

Object Oriented Programming

Syllabus

Unit I

Introduction to Internet and Web design - Working of Web - Roles of Front-end, Back-end and Full stack development - Web Server - Internet protocols - Web Hosting - HTML - HTML Tags - Create a simple Web Page - Marking up Text - Adding Links - Adding Images - SVG - Table Markup - Embedded Media - Web Based Forms - HTML Forms - CSS for Presentation - Basic Style Sheet - A CSS Style Primer - Using Style Classes - Using Style IDs - Formatting Text - Advanced Typography with CSS3 - Working with Margins, Padding, Alignment, and Floating. Responsive web design, Introduction to version control systems.

Unit II

Scripting - Javascript programming for Web Applications - Including Javascript in HTML - HTML5 Forms: Controls and Validation - Document Object Model - DOM manipulation and events - Event handlers

Unit III

Overview of UI Frameworks - User Interface Design Concepts in Web Development - Accessibility - Introduction to Electron - Overview of Electron - Setting up an Electron project - Creating a basic desktop application with Electron - Object communication - Introduction to other popular frameworks and libraries (e.g., Javafx, WPF, GTK).

User Interface Design

Master Time Table - CSE II Semester Master Time Table - CSE II Semester (B Section) Department of Computer Science & Engineering

Class: B.Tech CSE-B			Semester: II		Class Room: 511					
Class Advisors: 1. Dr. Kurnala Gowri Raghavendra Narayan Email: kg_narayana@anna.edu Mobile: +91 9705409393							2. Dr. Ankit Sachan Email: a_ankit@anna.edu Mobile: +91 838180129			
Time/Day	slot1 08:55-09:45	slot1 09:45-10:35	Short Break	slot3 10:45-11:35	slot4 11:35-12:25	slot5 12:25-01:15	slot6 02:10-03:00	Short Break	slot7 03:10-04:00	slot8 04:00-04:50
Monday	23MAT117 Linear Algebra LAB (BYOD)			LIBRARY	23PHY115 Modern Physics	LIBRARY	23MEE115 Manufacturing Practice LAB		23MEE115 Manufacturing Practice LAB	
Tuesday	23MAT116 Discrete Mathematics	23MAT117 Linear Algebra		23PHY115 Modern Physics	23CSE111 Object Oriented Programming	23CSE113 User Interface Design	Library / Sports		22ADM111 Glimpses of Glorious India	
Wednesday	23CSE111 OOP Lab			23MAT116 Discrete Mathematics	23CSE113 User Interface Design	22ADM111 Glimpses of Glorious India	COUNSELLING		Library / Sports	
Thursday	23MAT117 Linear Algebra	23CSE111 OOP		23MAT116 Discrete Mathematics	COUNSELLING	LIBRARY	23PHY115 Modern Physics (T)		Library / Sports	
Friday		23CSE113 UID LAB (BYOD)		23MAT117 Linear Algebra	23CSE111 OOP	Library / Sports	22ADM111 Glimpses of Glorious India (P)		23MAT116 Discrete Mathematics Lab	
Course Code		Course Name		L:T:P	Credits	Faculty Name		Assisting Faculty		
23MAT116		Discrete Mathematics		3-0-2	4	Dr. Shrivasa Rao Thota		-		
23MAT117		Linear Algebra		3-0-2	4	Mr. Sapeval Bixapathi		-		
23CSE111		Object Oriented Programming		3-0-2	4	Dr. Balaji TK		-		
23PHY115		Modern Physics		2-1-0	3	Dr. C R Kesavulu		-		
23CSE113		User Interface Design		2-0-2	3	Dr. Rajkumar Balchou		Mr. Barge Bhargadaj		
23MEE115		Manufacturing Practice		0-0-3	1	Dr. Sathies S		-		
22ADM111		Glimpses of Glorious India		2-0-1	2	Dr. Siva Panuganti		-		
Total					21	(15L + 1T + 12P = 28 Hrs)				

Explanation:

This HTML code creates a complete semester webpage that includes both a syllabus section and a timetable. The first part uses CSS Grid (.syllabus-grid) to display subjects as cards with images and names in a structured layout.

Each subject is shown inside a .subject-card with styling like borders, spacing, and centered text. The page also uses a background image and basic font styling for visual appeal.

The second part of the code builds a detailed timetable using tables, organizing subjects across days and time slots. CSS classes like .break and .counselling are used to highlight breaks and special sessions.

Another table (course-table) lists course details such as course code, credits, and faculty names. The code also uses features like colspan and rowspan to merge cells for better structure.

However, there are structural issues like multiple <head> and <body> tags and repeated styles. Overall, it demonstrates advanced HTML and CSS concepts like grid layout, tables, and styling.

LAB-4

4a) To understand the concept of forms and design a structured student registration form using HTML tables, including various input types such as text fields, radio buttons, checkboxes, file upload, and dropdown lists.

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>University Admissions Portal</title>

</head>

<body>

<h2 align="center">University Admissions Portal</h2>

<form action="#" method="POST" enctype="multipart/form-data">

<table align="center" border="1" cellpadding="10" cellspacing="0" width="60%">

  <tr>

    <th colspan="2">Student Registration Form</th>

  </tr>

  <tr>

    <td align="right"><b>Full Name:</b></td>

    <td><input type="text" name="fullname" placeholder="Enter full name" required></td>

  </tr>

  <tr>

    <td align="right"><b>Email Address:</b></td>

    <td><input type="email" name="email" placeholder="student@example.com"

required></td>

  </tr>

  <tr>

    <td align="right"><b>Phone Number:</b></td>

    <td><input type="tel" name="phone" placeholder="123-456-7890"></td>

  </tr>

</table>

</form>

</body>

</html>
```

```

</tr>
<tr>
  <td align="right"><b>Gender:</b></td>
  <td>
    <label><input type="radio" name="gender" value="male"> Male</label>
    <label><input type="radio" name="gender" value="female"> Female</label>
    <label><input type="radio" name="gender" value="other"> Other</label>
  </td>
</tr>
<tr>
  <td align="right"><b>Course Applying For:</b></td>
  <td>
    <label for="course">Select a Course:</label>
    <select id="course" name="course" required>
      <option value="">-- Select a Course --</option>
      <option value="cs">B.Tech CSE</option>
      <option value="aie">B.Tech AIE</option>
      <option value="aids">B.Tech AIDS</option>
      <option value="cce">B.Tech CCE</option>
    </select>
  </td>
</tr>
<tr>
  <td align="right"><b>Hobbies:</b></td>
  <td>
    <label><input type="checkbox" name="hobbies[]" value="reading">
Reading</label>
    <label><input type="checkbox" name="hobbies[]" value="sports"> Sports</label>
    <label><input type="checkbox" name="hobbies[]" value="music"> Music</label>
  </td>
</tr>

```

```
<label><input type="checkbox" name="hobbies[]" value="art"> Art</label>  
    </td>  
  
</tr>  
  
<tr>  
    <td align="right"><b>Date of Birth:</b></td>  
    <td><input type="date" name="dob" placeholder="MM/DD/YYYY" title="Enter your  
date of birth"></td>  
  
</tr>  
  
<tr>  
    <td align="right"><b>Upload Image:</b></td>  
    <td><input type="file" name="image" accept="image/*" title="Upload your image file"  
required></td>  
  
</tr>  
  
<tr>  
    <td align="right" valign="top"><b>Address:</b></td>  
    <td><textarea name="address" rows="4" cols="35" placeholder="Enter full  
address..."></textarea></td>  
  
</tr>  
  
<tr>  
    <td colspan="2" align="center">  
        <input type="submit" value="Submit Application">  
        &nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&~  
        <input type="reset" value="Clear Form">  
    </td>  
  
</tr>  
</table>  
</form>  
</body>  
</html>
```

OUTPUT: -

Name: MAHESWARAN Roll no: AV/SC.U4CSE25127 Sec: CSE-B

University Admissions Portal	
Student Registration Form	
Full Name:	Enter full name
Email Address:	student@example.com
Phone Number:	123-456-7890
Gender:	☐ Male ☐ Female ☐ Other
Course Applying For:	Select a Course: -- Select a Course --
Hobbies:	☐ Reading ☐ Sports ☐ Music ☐ Art
Date of Birth:	dd-mm-yyyy 📅
Upload Image:	Choose File No file chosen
Address:	Enter full address...
Submit Application Clear Form	

Explanation:

This HTML code creates a University Admissions Portal form for student registration. The `<form>` tag is used to collect user input, with `method="POST"` and `enctype="multipart/form-data"` to allow file uploads.

A table is used to neatly arrange form fields like name, email, phone number, and gender. Various input types such as `<input type="text">`, `<email>`, `<tel>`, and `<date>` are used to capture specific data.

Radio buttons are used for selecting gender, while checkboxes allow multiple hobbies to be chosen. A `<select>` dropdown is provided to choose the course.

The form also includes a file upload option for images and a `<textarea>` for entering the address. Finally, submit and reset buttons are added to send or clear the form data.

4b)To understand the concept of forms and design user login and registration webpages using HTML, and to display them within a single interface using iframe for better layout and user interaction.

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Integrated Auth Portal</title>

    <style>

        body {

            margin: 0;

            display: flex;

            justify-content: center;

            align-items: center;

            min-height: 100vh;

            background-color: #f0f2f5;

            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;

        }

        .portal-container {

            background-color: #ffffff;

            padding: 50px;

            display: flex;

            gap: 30px;

            border: 2px solid #0b3d52;

            box-shadow: 0 15px 35px rgba(0,0,0,0.2);

            border-radius: 4px;

        }

        iframe {
```

```

width: 320px;
height: 400px;
background-color: white;
border: 2px solid #000;
border-radius: 2px;
}
@media (max-width: 800px) {
  .portal-container {
    flex-direction: column;
    padding: 20px;
  }
}
</style>
</head>
<body>
<p class="student-info">Name: MAHESWARAN Roll no: AV.SC.U4CSE25127 Sec: CSE-
B</p>
<div class="portal-container">
  <iframe title="Login" srcdoc="
    <style>
      body { font-family: sans-serif; padding: 25px; text-align: center; color: #333; }
      input { width: 90%; padding: 10px; margin: 10px 0; border: 1px solid #ccc; border-
radius: 4px; }
      button { width: 95%; padding: 10px; background: #1a5f7a; color: white; border:
none; cursor: pointer; border-radius: 4px; margin-top: 10px; }
      button:hover { background: #144a5f; }
    </style>
    <h2>User Login</h2>
    <input type='text' placeholder='Username'>
    <input type='password' placeholder='Password'>
  </div>

```

```

        <button>login</button>
    "></iframe>

    <iframe title="Register" srcdoc="

        <style>

            body { font-family: sans-serif; padding: 25px; text-align: center; color: #333; }

            input { width: 90%; padding: 10px; margin: 8px 0; border: 1px solid #ccc; border-
radius: 4px; }

            button { width: 95%; padding: 10px; background: #1a5f7a; color: white; border:
none; cursor: pointer; border-radius: 4px; margin-top: 10px; }

            button:hover { background: #144a5f; }

        </style>

        <h2>User Registration</h2>

        <p>Create a new account</p>

        <input type='text' placeholder='Full Name'>

        <input type='email' placeholder='Email Address'>

        <input type='password' placeholder='Create Password'>

        <button>register</button><br><button>reset</button>

    "></iframe>

</div>

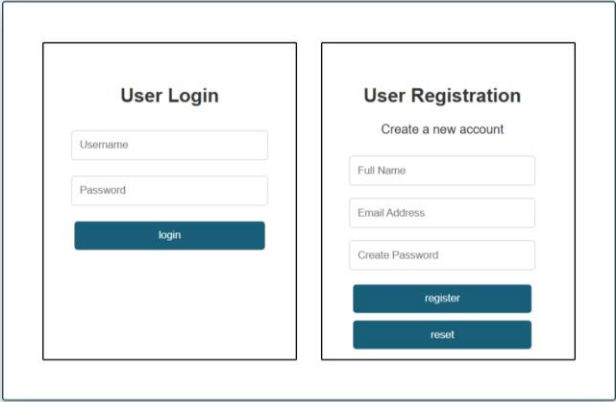
</body>

</html>

```

OUTPUT: -

Name: MAHESWARAN Roll no: AV5C.U4CSE25127 Sec: CSE-B



The screenshot displays a web application interface with two main sections side-by-side. On the left is the 'User Login' section, which includes input fields for 'Username' and 'Password', and a 'login' button. On the right is the 'User Registration' section, which includes input fields for 'Full Name', 'Email Address', and 'Create Password', along with 'register' and 'reset' buttons. The forms are styled with a light blue background and a subtle shadow effect.

Explanation:

This HTML code creates an Integrated Authentication Portal with login and registration sections displayed side by side. The `<style>` in the `<head>` uses Flexbox to center the layout and align both sections inside a `.portal-container`.

Two `<iframe>` elements are used to embed separate login and registration forms using the `srcdoc` attribute, which contains HTML and CSS directly. Each `iframe` includes input fields and buttons styled internally for user interaction.

The design is responsive using a media query (`@media`), which stacks the forms vertically on smaller screens. CSS properties like padding, borders, shadows, and hover effects improve the visual appearance.

The login form collects username and password, while the registration form collects name, email, and password. Overall, the code demonstrates embedding mini web pages inside a main page using `iframes` with styling.

LAB-5

5(A) To create a responsive card layout using HTML and CSS with three cards, each containing an image, title, and button.

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Responsive Card Layout</title>

<p class="student-info">Name: MAHESWARAN Roll no: AV.SC.U4CSE25127 Sec: CSE-
B</p>

<style>

body{

    font-family: Arial, Helvetica, sans-serif;

    margin:0;

    padding:0;

    background:#f4f4f4;

}

.card-container{

    display:flex;

    justify-content:center;

    flex-wrap:wrap;

    gap:20px;

    padding:40px;

}

.card{

    background:white;

    width:300px;

    border-radius:10px;
```

```
    overflow:hidden;

    box-shadow:0 4px 10px rgba(0,0,0,0.2);

    text-align:center;

    transition:transform 0.3s;
}

.card:hover{

    transform:scale(1.05);
}

.card img{

    width:100%;

    height:200px;

    object-fit:cover;


}

.card-content{

    padding:20px;
}

.card h3{

    margin:10px 0;
}

.card button{

    padding:10px 20px;

    border:none;

    background:#007BFF;

    color:white;

    border-radius:5px;

    cursor:pointer;

    transition:background 0.3s;
}

.card button:hover{
```

```

background:#0056b3;
}
@media(max-width:768px){
  .card-container{
    flex-direction:column;
    align-items:center;
  }
}
</style>
</head>
<body>
<div class="card-container">
  <div class="card">
    
    <div class="card-content">
      <h3>Royal Enfield Himalayan</h3>
      <p>The Royal Enfield Himalayan is an adventure touring motorcycle designed for both on-road and off-road riding. Equipped with a 411cc single-cylinder engine, it delivers reliable performance and torque. Its rugged build and long-travel suspension make it ideal for exploring diverse terrains. This bike combines classic styling with modern engineering for an unforgettable riding experience.</p>
      <a href="https://www.royalenfield.com/in/en/motorcycles/new-himalayan/">
        <button>buynow</button></a>
    </div>
  </div>
  <div class="card">
    
    <div class="card-content">
      <h3>Royal Enfield Meteor 350</h3>
      <p>The Royal Enfield Meteor 350 is a cruiser motorcycle that blends retro aesthetics with contemporary features. Powered by a 349cc air-cooled engine, it offers smooth and

```

effortless riding. Its comfortable seating position and relaxed ergonomics make it perfect for long city cruises. With modern technology like Bluetooth connectivity, it appeals to both classic and modern riders.</p>

<button>Buy Now</button>

</div>

</div>

<div class="card">

<div class="card-content">

<h3>Kawasaki Ninja 350</h3>

<p>The Kawasaki Ninja 350 is a sporty bike designed for urban performance and agility. Featuring a 348cc parallel-twin engine, it provides spirited acceleration and responsive handling. Its sleek and aggressive styling turns heads on the street. With advanced features like ABS and a digital display, it's both fun and practical for daily commuting.</p>

<button>Buy Now</button>

</div>

</div>

</div>

</body>

</html>

OUTPUT:

ame: MAHESWARAN Roll no: AV.SC.U4CSE25127 Sec: CSE-B



Royal Enfield Himalayan

The Royal Enfield Himalayan is an adventure touring motorcycle designed for both on-road and off-road riding. Equipped with a 411cc single-cylinder engine, it delivers reliable performance and torque. Its rugged build and long-travel suspension make it ideal for exploring diverse terrains. This bike combines classic styling with modern engineering for an unforgettable riding experience.

[buynow](#)



Royal Enfield Meteor 350

The Royal Enfield Meteor 350 is a cruiser motorcycle that blends retro aesthetics with contemporary features. Powered by a 349cc air-cooled engine, it offers smooth and effortless riding. Its comfortable seating position and relaxed ergonomics make it perfect for long city cruises. With modern technology like Bluetooth connectivity, it appeals to both classic and modern riders.

[Buy Now](#)



Kawasaki Ninja 350

The Kawasaki Ninja 350 is a sporty bike designed for urban performance and agility. Featuring a 348cc parallel-twin engine, it provides spirited acceleration and responsive handling. Its sleek and aggressive styling turns heads on the street. With advanced features like ABS and a digital display, it's both fun and practical for daily commuting.

[Buy Now](#)

On clicking buy now

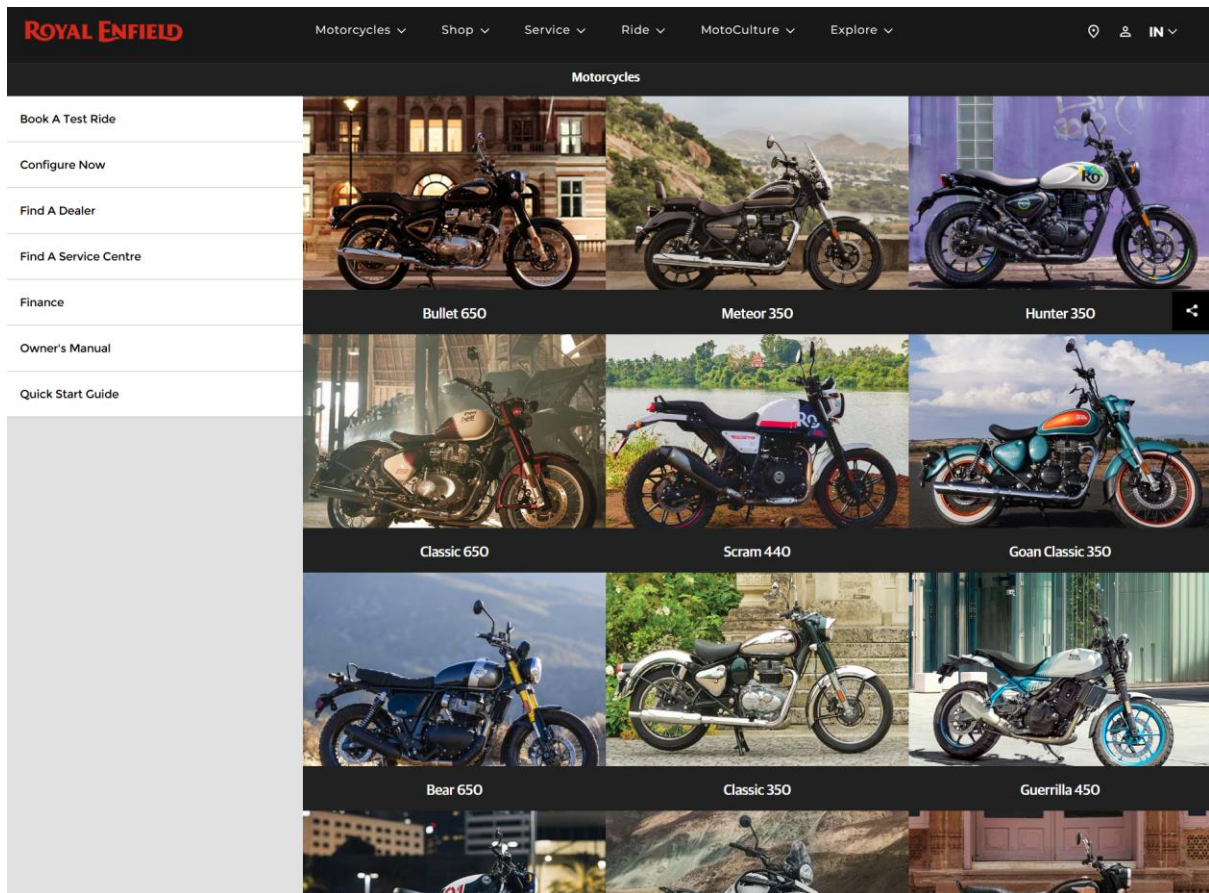
ROYAL ENFIELDMotorcycles ▾Shop ▾Service ▾Ride ▾MotoCulture ▾Explore ▾📍👤IN ▾



ROYAL ENFIELD
HIMALAYAN
ALL ROADS. NO ROADS.
#AllRoadsNoRoads

HIGHLIGHTS





Kawasaki
Let the Good Times Roll!

MY KAWASAKI | CART (0) | WISHLIST | TEST RIDE | LOCATE A DEALER | Search

MOTORCYCLE - ATV - SIDE X SIDE - NAV PTV - WATERCRAFT - ELECTRIFICATION - EXPERIENCE - GOVERNMENT & FLEET - SHOP - OWNER CENTER - RACING - RESEARCH TOOLS -

FORMULATED FOR GOOD TIMES Keep your engine running strong with KPO Oil Change Kits. Free shipping on orders \$100+ [SHOP KPO](#)

FINANCING AS LOW AS 2.95%
FOR 36 MONTHS**

[LEARN MORE](#)

**Model & year exclusions apply. May be combined with promotional retail financing rates. Incentives are available at participating Kawasaki dealerships. Offer valid through 06/30/2026. Subject to change without notice. Offer available on approved purchases of select new unregistered Kawasaki vehicles. **Kawasaki Motors (U.S.A.) does not offer or extend credit and does not require or make any discrimination of race.

How We Use Cookies

This website uses cookies to enable functionality, understand site performance, enhance our marketing efforts, and serve more relevant content and advertisements to you. If you would like to manage your cookie preferences, please click the Manage Cookies button. You can change your cookie preferences at any time by clicking "Do Not Sell or Share My Personal Information" in the footer of this site. By continuing to use this site, you agree to our Terms and Conditions and acknowledge our Privacy Policy.

[ACCEPT COOKIES](#)

[Manage Cookies](#)

[REJECT COOKIES](#)

Explanation:

This HTML code creates a responsive card layout webpage to display different motorcycles with images and details. The `<style>` section uses Flexbox (`display: flex`) to arrange multiple cards in a row and wrap them on smaller screens.

Each `.card` contains an image, title, description, and a “Buy Now” button linking to official websites. CSS properties like `box-shadow`, `border-radius`, and `transition` improve the visual design and add hover effects.

The `.card:hover` effect slightly enlarges the card for interactivity, while buttons change color on hover. A media query (`@media`) ensures the layout becomes vertical on smaller devices.

However, there is a syntax error in `.card img` (missing closing `}`), which may affect styling. Overall, the code demonstrates responsive design using Flexbox and interactive UI elements.

5(B) To design a mini website using iframe with navigation links (Home, About, Contact), where clicking on each link displays the respective content within the same page using internal or external CSS.

CODE:

Main file

```
<!DOCTYPE html>

<html lang="en">

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <head>

    <title>Our webpage</title>

    <link rel="stylesheet" href="styles.css">

  </head>

  <body >

    <p style="text-align: center; font-family: Arial, sans-serif; background-color:rgb(31, 2, 58);">This is a main page of our website.</p>

    <div style="text-align: center; font-family: Arial, sans-serif; background-color: #f0f0f0; display: flex; justify-content: center;">

      <div>

        <a href="home.html" style="margin-right: 20px;" target="iframe_a"><button style="background-color: #007BFF; color: white; border: none; padding: 10px 20px; border-radius: 5px; cursor: pointer;" onclick="this.style.backgroundColor='#0056b3';this.style.transform='scale(0.95)';set Timeout(() => {this.style.transform='scale(1)';this.style.backgroundColor='#007BFF';}, 150);">Home</button></a>

      </div>

      <div>

        <a href="About.html" style="margin-right: 20px;" target="iframe_a"><button style="background-color: #007BFF; color: white; border: none; padding: 10px 20px; border-radius: 5px; cursor: pointer;" onclick="this.style.backgroundColor='#0056b3';this.style.transform='scale(0.95)';set Timeout(() => {this.style.transform='scale(1)';this.style.backgroundColor='#007BFF';}, 150);">About</button></a>

      </div>

    </div>

  </body>

</html>
```

```

    </div>

    <div>

        <a href="contact.html" target="iframe_a"><button style="background-color:
#007BFF; color: white; border: none; padding: 10px 20px; border-radius: 5px; cursor:
pointer;"onclick="this.style.backgroundColor='#0056b3';this.style.transform='scale(0.95)';set
Timeout(() => {this.style.transform='scale(1)';this.style.backgroundColor='#007BFF';},
150);">Contact</button></a>

    </div>

</div>

<iframe name="iframe_a" height="400px" width="100%" title="Iframe
Example"></iframe>

</body>

</html>

```

Home.html

```

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Home Page</title>

</head>

<body>

<h1>Home Page</h1>

<p>Welcome to the home page.</p>

</body>

</html>

```

About.html

```

<!DOCTYPE html>

<html lang="en">

    <meta charset="UTF-8">

```

```

    <meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>

<title>Home</title>

<link rel="stylesheet" href="styles.css">
</head>

<body>

<h2>Welcome to My Web Development Hub</h2>

<p>

    This website is dedicated to exploring the essential building blocks of modern web
    development:

    <strong>HTML</strong>, <strong>CSS</strong>, and <strong>JavaScript</strong>.

</p>

<h3>What You'll Learn Here</h3>

<ul>

    <li>How <strong>HTML</strong> structures the content of a webpage.</li>
    <li>How <strong>CSS</strong> styles and enhances the visual presentation.</li>
    <li>How <strong>JavaScript</strong> adds interactivity and dynamic features.</li>

</ul>

<h3>Why These Technologies Matter</h3>

<p>

    Together, HTML, CSS, and JavaScript form the foundation of every website you visit.
    Whether you're building a simple personal page or a complex web application,
    these three technologies work hand in hand to bring your ideas to life.

</p>

<h3>Get Started</h3>

<p>

```

Use the navigation bar above to explore more about each technology and learn how they combine to create engaging, responsive, and interactive websites.

</p>

</body>

</html>

CONTACT.html

<!DOCTYPE html>

<html lang="en">

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<head>

<title>Contact</title>

<link rel="stylesheet" href="styles.css">

</head>

<body>

<h2>Contact Us</h2>

<h3>Contact Details</h3>

<p>Email: mrx@email.com</p>

<p>Phone: +91-9876543210</p>

<p>Address: Amaravati, Andhra Pradesh, India</p>

<h3>Send Us a Message</h3>

<form>

<label for="name">Name:</label>

<input type="text" id="name" name="name" placeholder="Your name">

<label for="email">Email:</label>


```
<input type="email" id="email" name="email" placeholder="Your email"><br><br>
```

```
<label for="message">Message:</label><br>
```

```
<textarea id="message" name="message" rows="5" placeholder="Your  
message"></textarea><br><br>
```

```
<input type="submit" value="Send">
```

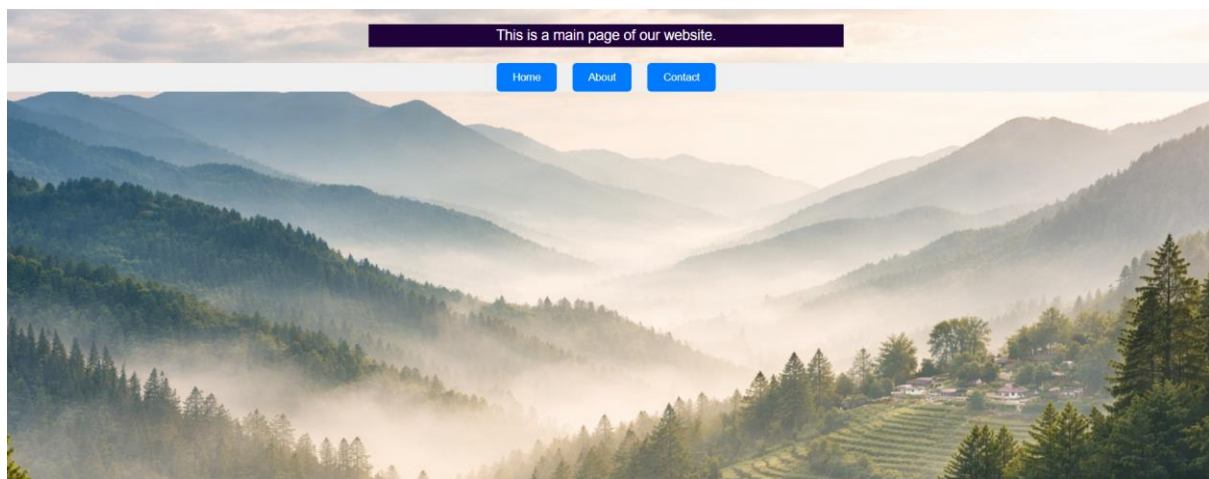
```
</form>
```

```
</body>
```

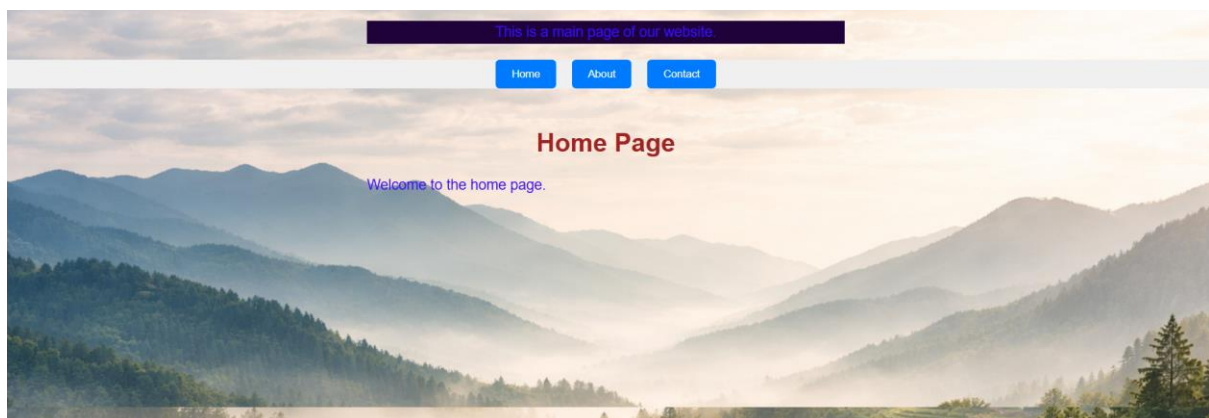
```
</html>
```

OUTPUT:

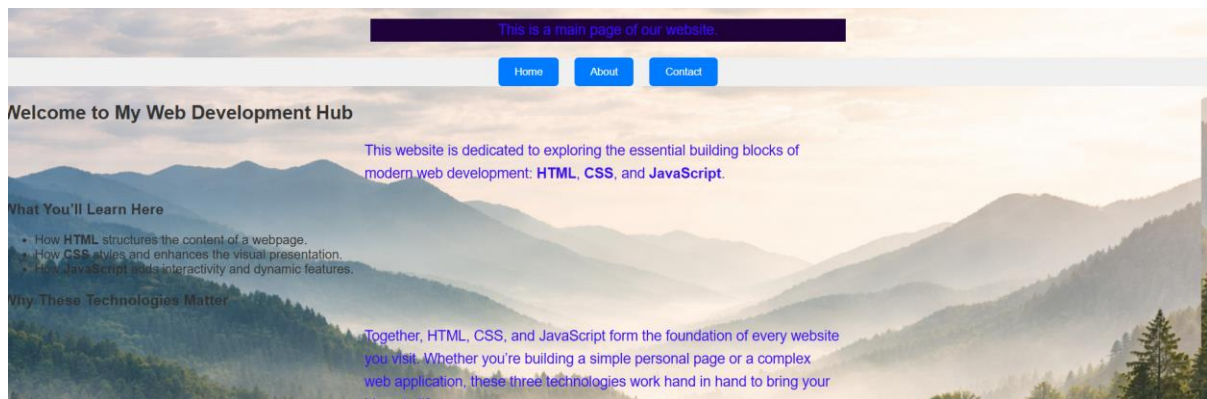
MAIN.HTML



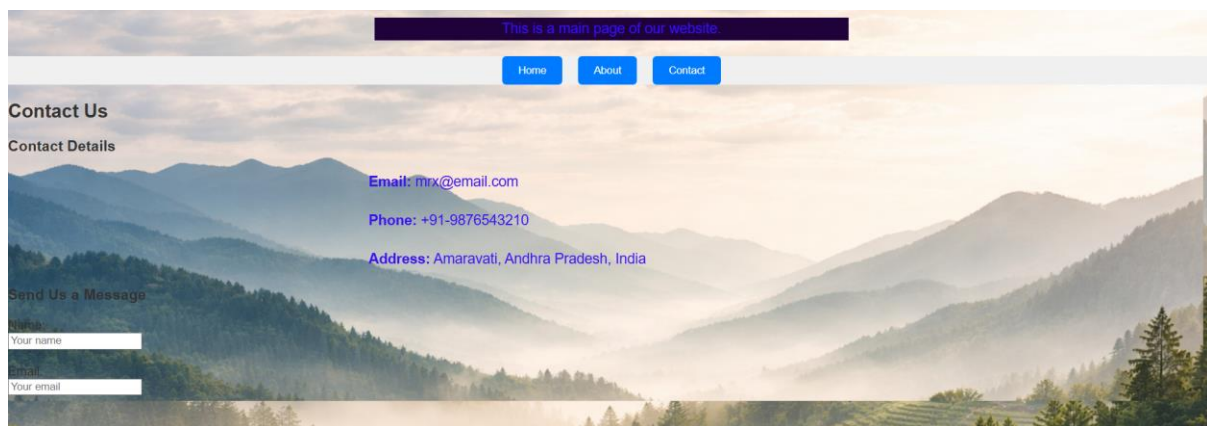
HOME.HTML



ABOUT.HTML



CONTACT.HTML



Explanation:

This project creates a multi-page website using an iframe-based navigation system. The main file contains navigation buttons (Home, About, Contact) that load different pages into an `<iframe>` using the target attribute.

Inline CSS is used to style the navigation bar and buttons, and JavaScript is added inside onclick to create a click animation effect. The `<iframe>` acts as a display area where selected pages are shown without reloading the main page.

The home.html file displays a simple welcome message, while About.html explains HTML, CSS, and JavaScript concepts. The contact.html page includes contact details and a basic form for user input.

All pages follow basic HTML structure and some link to an external stylesheet (styles.css) for styling. Overall, the code demonstrates page navigation, iframe usage, and basic interactivity in web development.

5(C) To design an Amrita login page using HTML and CSS that displays different campuses, where clicking on each campus shows its details on the same page.

Main.html

```
<!doctype html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Amrita Campus Login</title>

<style>

body{

    margin:0;

    font-family:Arial;

    background-color:#a9445b;

}

.header{

    background:#7a1f33;

    color:white;

    padding:25px;

    text-align:center;

}

.campus-section{

    padding:40px;

    display:grid;

    grid-template-columns: repeat(3, 1fr);

    gap:25px;

}

.card{

    border-radius:10px;
```

```
        overflow:hidden;
    }
    .card img{
        width:100%;
        height:200px;
        object-fit:cover;
    }
    .card a{
        text-decoration:none;
        color:white;
    }
    .overlay{
        position:relative;
        margin-top:-40px;
        padding:10px;
        text-align:center;
        font-weight:bold;
    }
    iframe{
        width:80%;
        height:300px;
        border:none;
        border-radius:10px;
        background:white;
        margin:40px auto;
        display:block;
    }
</style>
</head>
```

```

<body>
<div class="header">
  <h1>Amrita Vishwa Vidyapeetham</h1>
  <p>Campus Login Portal</p>
</div>
<div class="campus-section">
  <div class="card">
    <a href="coimbatore-login.html" target="loginFrame">
      
      <div class="overlay">Coimbatore</div>
    </a>
  </div>
  <div class="card">
    <a href="bangalore-login.html" target="loginFrame">
      
      <div class="overlay">Bangalore</div>
    </a>
  </div>
  <div class="card">
    <a href="amritapuri-login.html" target="loginFrame">
      
      <div class="overlay">Amritapuri</div>
    </a>
  </div>
  <div class="card">
    <a href="chennai-login.html" target="loginFrame">
      
      <div class="overlay">Chennai</div>
    </a>
  </div>

```

```

</div>

<div class="card">

  <a href="amaravati-login.html" target="loginFrame">

    <div class="overlay">Amaravati</div>

  </a>

</div>

<div class="card">

  <a href="kochi-login.html" target="loginFrame">

    <div class="overlay">Kochi</div>

  </a>

</div>

</div>

<iframe name="loginFrame" title="Campus Login Form"></iframe>

</body>

</html>

```

Coimbatore Campus Login

```

<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Coimbatore Campus Login</title>

  <link rel="stylesheet" href="styles.css">

</head>

<body>

<div class="container">

<h2>Coimbatore Campus Login</h2>

```

```
<form>

  <input type="text" placeholder="Enter ID" class="input-field"><br>

  <input type="password" placeholder="Enter Password" class="input-field"><br>

  <button type="submit" class="submit-btn">

    Submit

  </button>

</form>

</div>

</body>

</html>
```

Bangalore Campus Login

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Bangalore Campus Login</title>

  <link rel="stylesheet" href="styles.css">

</head>

<body>

  <div class="container">

    <h2>Bangalore Campus Login</h2>

    <form>

      <input type="text" placeholder="Enter ID" class="input-field"><br>

      <input type="password" placeholder="Enter Password" class="input-field"><br>

      <button type="submit" class="submit-btn">

        Submit

      </button>

    </form>

  </div>

</body>

</html>
```

```
</form>
```

```
</div>
```

```
</body>
```

```
</html>
```

Amritapuri Campus Login

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Amritapuri Campus Login</title>
```

```
  <link rel="stylesheet" href="styles.css">
```

```
</head>
```

```
<body>
```

```
<div class="container">
```

```
<h2>Amritapuri Campus Login</h2>
```

```
<form>
```

```
  <input type="text" placeholder="Enter ID" class="input-field"><br>
```

```
  <input type="password" placeholder="Enter Password" class="input-field"><br>
```

```
  <button type="submit" class="submit-btn">
```

```
    Submit
```

```
  </button>
```

```
</form>
```

```
</div>
```

```
</body>
```

```
</html>
```

Chennai Campus Login

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Chennai Campus Login</title>

  <link rel="stylesheet" href="styles.css">

</head>

<body>

<div class="container">

<h2>Chennai Campus Login</h2>

<form>

  <input type="text" placeholder="Enter ID" class="input-field"><br>

  <input type="password" placeholder="Enter Password" class="input-field"><br>

  <button type="submit" class="submit-btn">

    Submit

  </button>

</form>

</div>

</body>

</html>

```

Amaravati Campus Login

```

<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Amaravati Campus Login</title>

  <link rel="stylesheet" href="styles.css">

</head>

<body>

```

```

<div class="container">
<h2>Amaravati Campus Login</h2>
<form>
  <input type="text" placeholder="Enter ID" class="input-field"><br>
  <input type="password" placeholder="Enter Password" class="input-field"><br>
  <button type="submit" class="submit-btn">
    Submit
  </button>
</form>
</div>
</body>
</html>

```

Kochi Campus Login

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Kochi Campus Login</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
<div class="container">
<h2>Kochi Campus Login</h2>
<form>
  <input type="text" placeholder="Enter ID" class="input-field"><br>
  <input type="password" placeholder="Enter Password" class="input-field"><br>
  <button type="submit" class="submit-btn">
    Submit

```

</button>

</form>

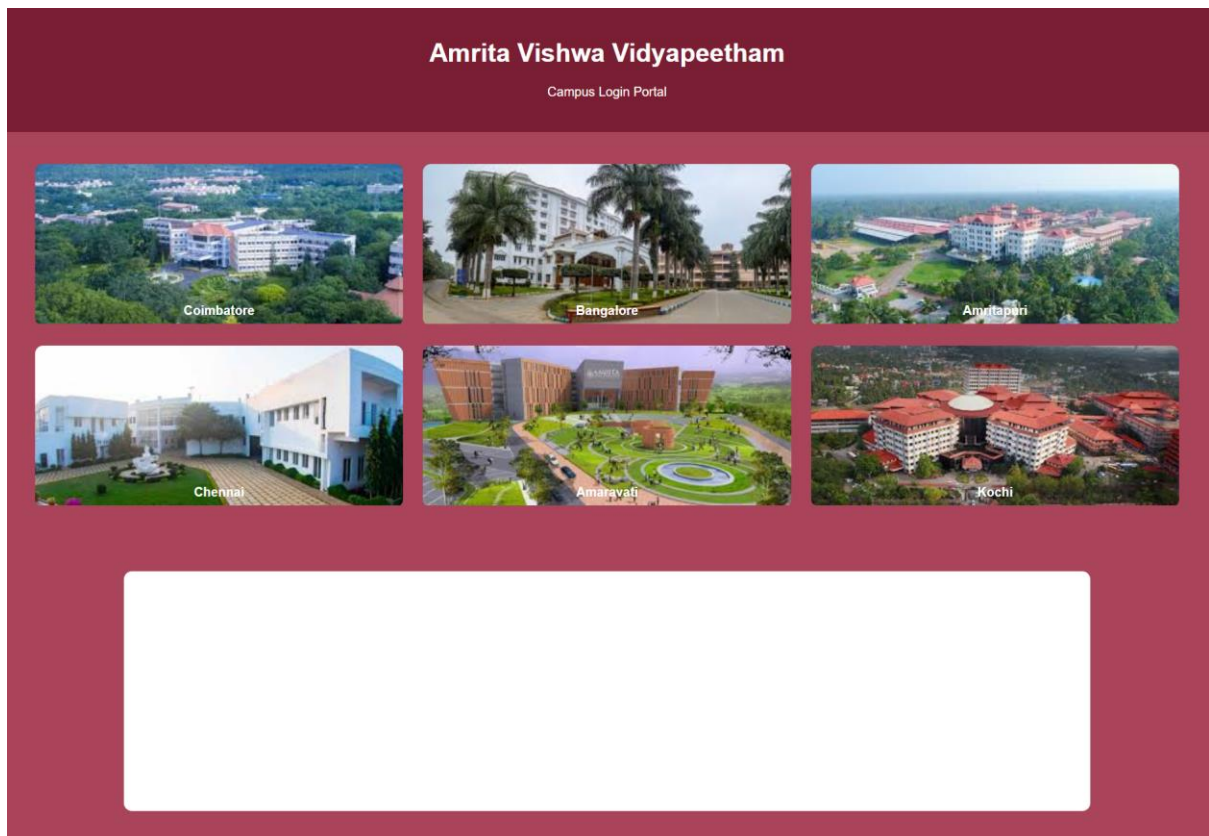
</div>

</body>

</html>

OUTPUT:

Main page



Coimbatore

Amrita Vishwa Vidyapeetham
Campus Login Portal



Coimbatore Campus Login



Bangalore

Amrita Vishwa Vidyapeetham
Campus Login Portal



Bangalore Campus Login



Amritapuri

Amrita Vishwa Vidyapeetham

Campus Login Portal



Coimbatore



Bangalore



Amritapuri



Chennai



Amaravati



Kochi

Amritapuri Campus Login

Enter ID

Enter Password



Kochi

Amrita Vishwa Vidyapeetham

Campus Login Portal



Coimbatore



Bangalore



Amritapuri



Chennai



Amaravati



Kochi

Kochi Campus Login

Enter ID

Enter Password



Chennai

Amrita Vishwa Vidyapeetham
Campus Login Portal



Chennai Campus Login



Amaravati

Amrita Vishwa Vidyapeetham
Campus Login Portal



Amaravati Campus Login

Explanation:

This project creates a Campus Login Portal where users can select different campuses and access their login forms. The main page uses a grid layout (display: grid) to show campus cards with images and names.

Each card is clickable and linked using `<a>` tags, which load the respective campus login page into an `<iframe>` using the target attribute. The header section displays the university name and portal title with styled background colors.

CSS is used to design the layout, including card styling, image display, spacing, and overall page appearance. The `.overlay` class is used to show campus names on top of images.

The `<iframe>` at the bottom acts as a container where selected login pages are displayed without reloading the main page. Each campus (Coimbatore, Bangalore, Chennai, etc.) has its own separate HTML login page.

These login pages contain simple forms with input fields for ID and password, along with a submit button. All pages use consistent structure and styling through an external CSS file (`styles.css`).

LAB-6

(6) Design a responsive e-commerce application using the grid layout. This application should contain different categories of products such as electronics, clothing, home-appliances, books. Based on the category we select corresponding products should be displayed on the sidebar.

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Amrita Sales</title>

<style>

body{

font-family:'Times New Roman', Times, serif;

margin:0;

}

header{

background:#952732a3;

color:rgb(255, 246, 246);

padding:15px;

font-size:22px;

}

.container{

display:flex;

}

x

.sidebar{

width:220px;
```

```
background:#c61d2d;
color:white;
padding:20px;
height:500px;
}
.sale-banner{
background:#ffe5e5;
color:#c61d2d;
text-align:center;
padding:10px;
font-size:20px;
font-weight:bold;
}
.sidebar label{
display:block;
margin:10px 0;
cursor:pointer;
}
input{
display:none;
}
.products{
display:grid;
grid-template-columns:repeat(2,1fr);
gap:20px;
padding:20px;
flex:1;
}
.card{
```

```
border:1px solid #ddd;
padding:10px;
background:#f8f8f8;
}
.card img{
width:100%;
height:180px;
object-fit:cover;
}
.price{
font-weight:bold;
margin-top:5px;
}
button{
background:#c61d2d;
color:white;
border:none;
padding:6px 12px;
margin-top:5px;
cursor:pointer;
}
.products .card {
display: none;
}
#all:checked ~ .container .card,
#electronics:checked ~ .container .electronics,
#clothing:checked ~ .container .clothing,
#books:checked ~ .container .books,
#home:checked ~ .container .home {
```

```

    display: block;
}
</style>
</head>
<body>
<header>Amrita Sales</header>
<input type="radio" name="cat" id="all" checked>
<input type="radio" name="cat" id="electronics">
<input type="radio" name="cat" id="clothing">
<input type="radio" name="cat" id="home">
<input type="radio" name="cat" id="books">
<div class="sale-banner">
🎉 Diwali Sale - 50% OFF on Selected Products 🎉
</div>
<div class="container">
<div class="sidebar">
<h3>Filter By Categories</h3>
<label for="all">> All</label>
<label for="electronics">> Electronics</label>
<label for="clothing">> Clothing</label>
<label for="home">> Home Goods</label>
<label for="books">> Books</label>
</div>
<div class="products">
<div class="card books">

<h4>Little Can We Be Strangers Again?</h4>
<p>Classic story book</p>
<p class="price">₹350</p>

```

```
<button>Add to Cart</button>

</div>

<div class="card clothing">



<h4>T Shirt</h4>

<p>Comfortable cotton t-shirt</p>

<p class="price">₹750</p>

<button>Add to Cart</button>

</div>

<div class="card electronics">



<h4>Mac neo </h4>

<p>High performance laptop</p>

<p class="price">₹90000</p>

<button>Add to Cart</button>

</div>

<div class="card electronics">



<h4>Samsung S24</h4>

<p>Android smartphone</p>

<p class="price">₹100000</p>

<button>Add to Cart</button>

</div>

<div class="card home">



<h4>Microwave</h4>

<p>Home appliance</p>

<p class="price">₹8000</p>

<button>Add to Cart</button>
```

</div>

</div>

</div>

</body>

</html>

OUTPUT:

Amrita Sales

🎉 Diwali Sale - 50% OFF on Selected Products 🎉

Filter By Categories

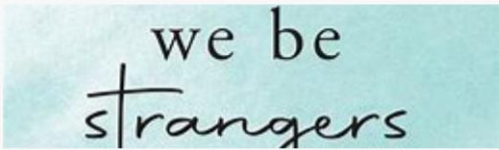
> All

> Electronics

> Clothing

> Home Goods

> Books



Little Can We Be Strangers Again?

Classic story book

₹350

Add to Cart

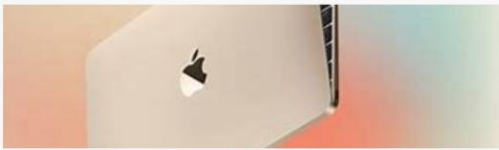


T Shirt

Comfortable cotton t-shirt

₹750

Add to Cart



Mac neo

High performance laptop

₹90000

Add to Cart



Samsung S24

Android smartphone

₹100000

Add to Cart



Microwave

Home appliance

₹8000

Add to Cart

Explanation:

This HTML code creates a simple e-commerce webpage called “Amrita Sales” with category-based product filtering. The `<style>` section is used to design the layout, including header, sidebar, product grid, and cards.

A header displays the website title, and a sale banner highlights a Diwali discount offer. The layout uses Flexbox (`display: flex`) to divide the page into a sidebar and a product section.

The sidebar contains category filters (All, Electronics, Clothing, Home, Books) implemented using hidden radio buttons and labels. CSS selectors are used to show or hide products based on the selected category.

The `.products` section uses CSS Grid to display items in a two-column layout. Each product is placed inside a `.card` with an image, title, description, price, and “Add to Cart” button.

Images are styled using `object-fit: cover` to maintain proper proportions. Buttons are styled for better appearance and interaction.

The filtering works without JavaScript by using the `:checked` selector and general sibling (`~`) combinator in CSS.

LAB-7

7) To create the repository in GIT Hub and push or upload files of all the programs that are done to create different webpages and websites.

Procedure:

1. Create a GitHub account Go to github.com, click Sign Up, and register using your email, username, and password.
2. Install Git locally Download Git from git-scm.com and install it on your computer. Verify installation with 'git --version' in terminal.
3. Configure Git user details Set your username and email with 'git config --global user.name "Your Name"' and 'git config --global user. Email "you@example.com"'.
4. Create a new repository On GitHub, click New Repository, give it a name (e.g., Lab7-Webpages), choose public/private, and click Create.
5. Initialize Git in project folder Open terminal in your project folder and run 'git init' to start version control.
6. Add and commit files Use 'git add.' to stage all files, then 'git commit -m "Initial commit"' to save changes.
7. Connect to GitHub repository Copy the repository URL from GitHub and run 'git remote add origin <URL>' in your terminal.
8. Push files to GitHub Run 'git push -u origin main' (or master if default) to upload your files to the repository.

Commands that I used to push my files are:

1. `git init` Initializes a new Git repository in your current folder. It creates a hidden `.git` directory where Git stores all version history.
2. `git add .` Stages all files in the current directory (the `.` means “everything here”). This tells Git which files you want to include in the next commit.
3. `git commit -m "First commit"` Saves a snapshot of the staged files into the repository with a message ("First commit"). The message should describe what you changed or added.
4. `git branch -M main` Renames the current branch to `main`. GitHub uses `main` as the default branch name now (instead of `master`).
5. `git remote add origin <PASTE_YOUR_URL_HERE>` Links your local repository to a remote repository on GitHub. Replace `<PASTE_YOUR_URL_HERE>` with the HTTPS or SSH URL of your GitHub repo.

6. `git push -u origin main` Pushes your local main branch to the remote repository named origin (GitHub). The `-u` sets up tracking so future git push commands know where to send changes.

Updating Files Later

- `git add .` Stage new or modified files again.
- `git commit -m "RANDOM"` Commit those changes with a message (here "RANDOM" is just a placeholder — use something meaningful like "Updated CSS").
- `git push origin main` Pushes the new commit(s) to GitHub.

The screenshot shows a GitHub repository page for a user named maheswaran-1201. The repository is named UID-lab_--25127 and is private. The page displays a list of commits, with the most recent one being an update to the file WEEK 5. The commit history shows a series of initial commits for various files (WEEK 1 through WEEK 8, .gitattributes, and .hintrc) and updates to WEEK 5 and WEEK 6. The right sidebar shows the repository's metadata, including 0 stars, 0 forks, and 0 releases. The bottom of the page shows the contributors section with 1 contributor listed.

Commit	Author	Message	Time
ae0db66	maheswaran-1201	updated file	2 hours ago
...	...	...	...
...	...	Initial commit	last month
...	...	updated file	yesterday
...	...	Initial commit	last month
...	...	Initial commit	last month
...	...	Initial commit	last month
...	...	Initial commit	last month
...	...	Initial commit	last month
...	...	updated file	3 hours ago
...	...	update	20 hours ago
...	...	Initial commit	last month
...	...	Initial commit	last month

LAB-8

8a) create a webpage that displays a square that continuously rotates 360 using css animation.

CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    body {
      padding: 20px;
    }
    .square {
      width: 140px;
      height: 140px;
      background-color: rgb(204, 20, 57);
      animation: rotateSquare 2s linear infinite;
    }
    @keyframes rotateSquare {
      from {
        transform: rotate(0deg);
      }
      to {
        transform: rotate(360deg);
      }
    }
  </style>
```

```
</head>

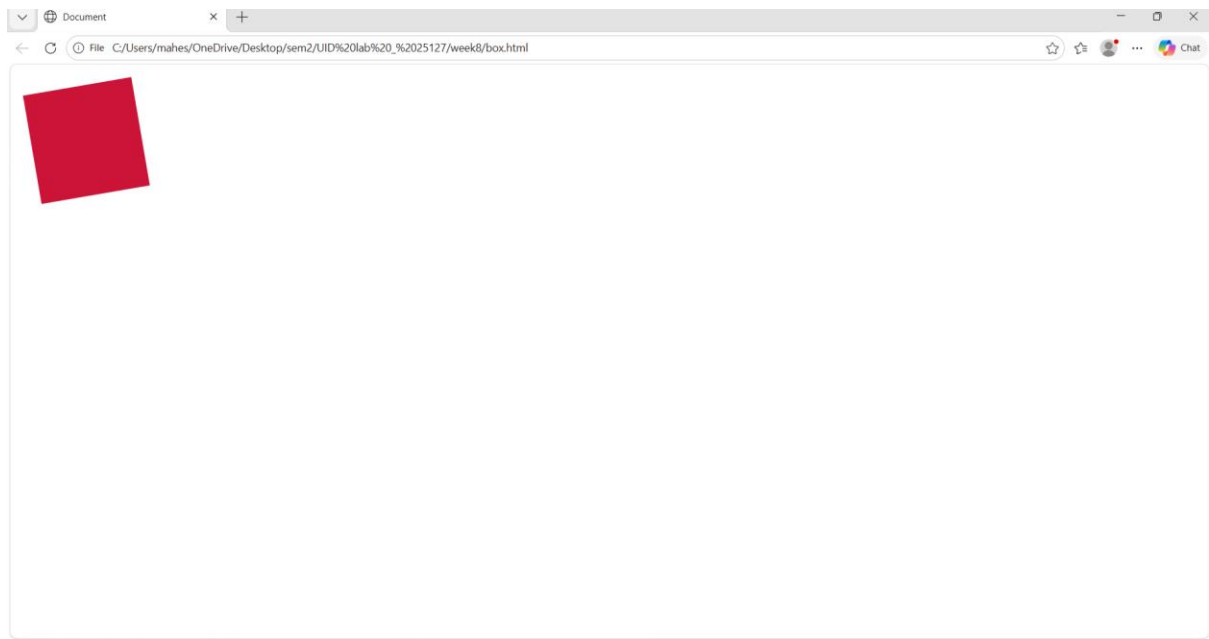
<body>

  <div class="square"></div>

</body>

</html>
```

OUTPUT: -



Explanation:

This HTML code creates a simple animation of a rotating square using CSS. The `<style>` section defines the design and animation properties for the square element.

The body has padding added to give space around the content. A `<div>` with class `.square` is used to create a box with fixed width and height.

The square is given a background color and styled as a solid block. The animation property is applied to make the square rotate continuously.

The `@keyframes rotateSquare` defines the animation steps, rotating the square from 0deg to 360deg. The 2s duration means the rotation completes in 2 seconds.

The linear timing function ensures smooth and constant speed, and infinite makes the animation repeat forever.

The `transform: rotate()` property is responsible for the rotation effect. Overall, the code demonstrates how to create basic animations using CSS keyframes.

8b) create a ball that moves up and down continuously using css animations.

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Document</title>

  <style>

    body {

      padding: 200px;

    }

    .ball {

      width: 100px;

      height: 100px;

      border-radius: 50%;

      background: #ef4444;

      animation: bounce 1.2s ease-in-out infinite;

    }

    @keyframes bounce {

      0%, 100% {

        transform: translateY(0);

      }

      50% {

        transform: translateY(-120px);

      }

    }

  </style>

</head>
```

```
<body>
  <div class="ball"></div>
</body>
</html>
```

OUTPUT:



Explanation:

This HTML code creates a bouncing ball animation using CSS. The `<style>` section defines the layout and animation behavior for the ball element.

The body has padding to position the ball away from the edges of the screen. A `<div>` with class `.ball` is used to create a circular shape.

The ball is styled with equal width and height and `border-radius: 50%` to make it perfectly round. A red background color is applied for visibility.

The animation property applies the bounce animation with a duration of 1.2 seconds. The ease-in-out timing function makes the movement smooth, and infinite repeats it continuously.

The `@keyframes bounce` defines the motion of the ball. At 0% and 100%, the ball stays at its original position.

At 50%, the ball moves upward using `transform: translateY(-120px)`. This creates a bouncing effect.

8c) create a circle that move horizontally and rotates at the same time.

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Document</title>

  <style>

    body {

      padding: 100px;

    }

    .circle {

      width: 100px;

      height: 100px;

      border-radius: 75%;

      background: linear-gradient(45deg, #9674bc, #832659);

      animation: moveAndRotate 2s linear infinite alternate;

    }

    @keyframes moveAndRotate {

      from {

        transform: translateX(0) rotate(0deg);

      }

      to {

        transform: translateX(320px) rotate(360deg);

      }

    }

  </style>

</head>
```

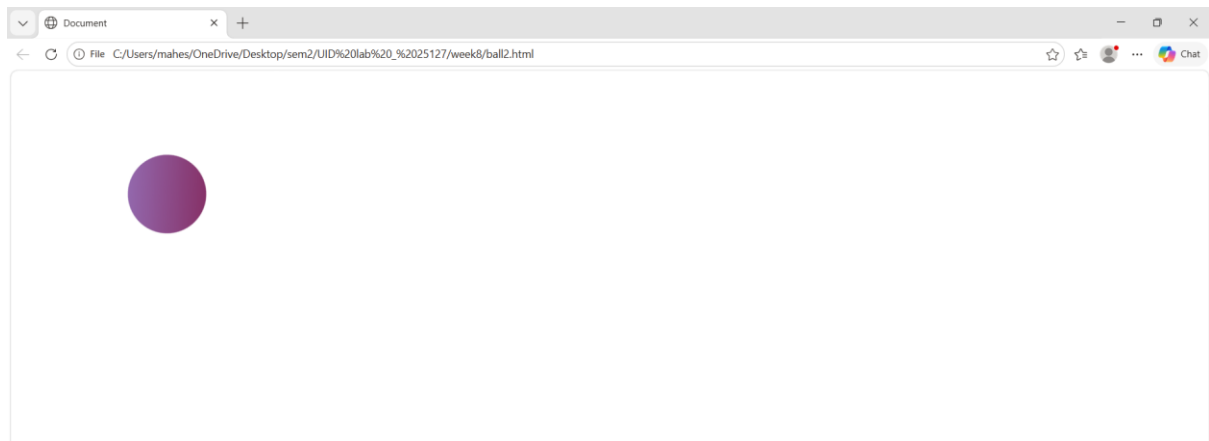
```
<body>

  <div class="circle"></div>

</body>

</html>
```

OUTPUT:



Explanation:

This HTML code creates an animated moving and rotating circle using CSS. The `<style>` section defines the appearance and animation behaviour of the circle.

The body has padding to position the element away from the edges. A `<div>` with class `.circle` is used to create the shape.

The circle is styled with equal width and height and border-radius to give it a rounded shape. A linear-gradient background adds a smooth color transition.

The animation property applies the `moveAndRotate` animation with a duration of 2 seconds. The linear timing ensures constant speed, and infinite alternate makes it move back and forth continuously.

The `@keyframes moveAndRotate` defines the animation from starting position to end position. The element moves horizontally using `translateX(320px)`.

At the same time, it rotates 360 degrees using `rotate(360deg)`. Overall, the code demonstrates combining movement and rotation animations in CSS.

Lab-9

Create a webpage using html, CSS, java script based on the given conditions:

a) write a JS program to determine if a number is even or odd

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>even or odd using javascript</title>

</head>

<body>

    <button onclick="showInputBox()">Enter Number</button>

    <div id="inputSection" style="display:none; margin-top: 20px;">

        <input type="number" id="numberInput" placeholder="Enter a number" />

        <button onclick="checkEvenOdd()">Check</button>

        <p id="result"></p>

    </div>

<script>

    function showInputBox(){

        document.getElementById("inputSection").style.display = "block";

    }

    function checkEvenOdd(){
```

```
let num = document.getElementById("numberInput");
let resultDiv = document.getElementById("result");
if (num === "") {
    resultDiv.textContent = "Please enter a number!";
    resultDiv.style.color = "red";
    return;
}
num = parseInt(num);
if (num % 2 === 0) {
    resultDiv.textContent = num + " is an Even number.";
    resultDiv.style.color = "green";
} else {
    resultDiv.textContent = num + " is an Odd number.";
    resultDiv.style.color = "blue";
}
}
</script>
</body>
</html>
```

OUTPUT:

4 is an Even number.

Explanation:

This HTML code creates a simple Even or Odd Number Checker using JavaScript. The webpage contains a button that allows the user to enter a number and check whether it is even or odd. The `<body>` section contains a button labeled “Enter Number.” When this button is clicked, the `showInputBox()` JavaScript function is called, which makes the hidden input section visible. A `<div>` element with the id `inputSection` contains a number input field, a “Check” button, and a paragraph element to display the result. Initially, this section is hidden using `display:none`. The input field is created using `<input type="number">`, which allows only numeric values to be entered. The placeholder text guides the user to enter a number. When the “Check” button is clicked, the `checkEvenOdd()` function is executed. This function reads the value entered by the user and checks whether the input field is empty. If no number is entered, an error message is displayed in red color. The entered value is converted into an integer using `parseInt()`. The program then uses the modulus operator `%` to determine whether the number is divisible by 2. If the remainder is 0, the number is even, and a message is displayed in green color. Otherwise, the number is odd, and the result is displayed in blue color.

B)write a JS program to determine if a number is prime or not

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>prime or not</title>

  <style>

    #inputSection {

      display: none;

      margin-top: 20px;

    }

  </style>

</head>

<body>

  <button onclick="showInputBox()">Enter Number</button>

  <div id="inputSection">

    <input type="number" id="numberInput" placeholder="Enter a number" />

    <button onclick="checkPrime()">Check</button>

    <p id="result"></p>

  </div>

  <script>

    function showInputBox(){

      document.getElementById("inputSection").style.display = "block";

    }

    function checkPrime(){

      let num = document.getElementById("numberInput").value;

      let resultDiv = document.getElementById("result");
```

```

    if (num === "") {
        resultDiv.textContent = "Please enter a number!";
        resultDiv.style.color = "red";
        return;
    }
    num = parseInt(num);
    if (isPrime(num)) {
        resultDiv.textContent = num + " is a Prime number.";
        resultDiv.style.color = "green";
    } else {
        resultDiv.textContent = num + " is not a Prime number.";
        resultDiv.style.color = "blue";
    }
}

function isPrime(n) {
    if (n <= 1) return false;
    if (n <= 3) return true;
    if (n % 2 === 0 || n % 3 === 0) return false;
    for (let i = 5; i * i <= n; i += 6) {
        if (n % i === 0 || n % (i + 2) === 0) return false;
    }
    return true;
}

</script>
</body>
</html>

```

OUTPUT:

Enter Number

18 is not a Prime number.

Explanation:

This HTML code creates a simple Prime Number Checker using JavaScript. The webpage contains a button that displays an input section when clicked. The input section includes a number input field, a “Check” button, and a paragraph to display the result.

When the user enters a number and clicks the “Check” button, the `checkPrime()` function is executed. The program first checks whether the input field is empty and displays an error message if no value is entered. The entered value is then converted into an integer using `parseInt()`.

The `isPrime()` function checks whether the number is prime or not using conditional statements and a loop. If the number has factors other than 1 and itself, it is not prime. The result is displayed on the webpage with different colors for prime and non-prime numbers.

c)write a JS program to find the factorial of a number

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>factorial</title>

  <style>

    #inputSection {

      display: none;

      margin-top: 20px;

    }

  </style>

</head>

<body>

  <div>

    <button onclick="showInputBox()">Enter Number</button>

    <div id="inputSection">

      <input type="number" id="numberInput" placeholder="Enter a number" />

      <button onclick="calculateFactorial()">Calculate</button>

      <p id="result"></p>

    </div>

    <script>

      function showInputBox(){

        document.getElementById("inputSection").style.display = "block";

      }

      function calculateFactorial(){
```

```
let num = document.getElementById("numberInput").value;
let resultDiv = document.getElementById("result");

if (num === "") {
    resultDiv.textContent = "Please enter a number!";
    resultDiv.style.color = "red";
    return;
}

num = parseInt(num);
if (num < 0) {
    resultDiv.textContent = "Factorial is not defined for negative numbers.";
    resultDiv.style.color = "red";
    return;
}

let factorial = 1;
for (let i = 1; i <= num; i++) {
    factorial *= i;
}

resultDiv.textContent = "Factorial of " + num + " is " + factorial + ".";
resultDiv.style.color = "green";
}
</script>
</div>
</body>
</html>
```

OUTPUT:

Enter Number

Calculate

Factorial of 7 is 5040.

Explanation:

This HTML code creates a simple Factorial Calculator using JavaScript. The webpage contains a button that displays an input section when clicked. The input section includes a number input field, a “Calculate” button, and a paragraph to display the result. When the user enters a number and clicks the “Calculate” button, the calculateFactorial() function is executed. The program first checks whether the input field is empty and displays an error message if no value is entered. The entered value is then converted into an integer using parseInt(). The program also checks whether the entered number is negative because factorial is not defined for negative numbers. If a negative number is entered, an error message is displayed in red color. A for loop is used to calculate the factorial by multiplying numbers from 1 up to the entered number. The final factorial value is then displayed on the webpage in green color.

d)write a JS program to find the given number is Armstrong number or not.

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Armstrong or not</title>

</head>

<body>

  <script>

    let num = prompt("Enter a number:");

    let sum = 0;

    let temp = num;

    while (temp > 0) {

      let digit = temp % 10;

      sum += digit ** 3;

      temp = Math.floor(temp / 10);

    }

    if (sum == num) {

      alert(num + " is an Armstrong number.");

    } else {

      alert(num + " is not an Armstrong number.");

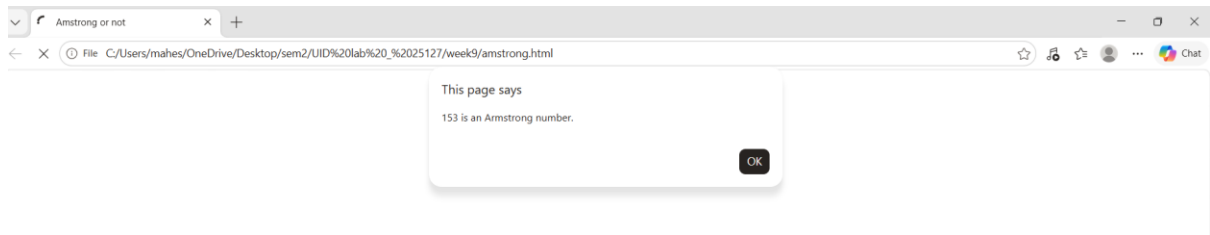
    }

  </script>

</body>

</html>
```

OUTPUT:



Explanation:

This HTML code checks whether a number is an Armstrong number or not using JavaScript. The program uses the `prompt()` function to take a number as input from the user. Variables `sum` and `temp` are used to store the calculated result and temporary value of the number.

A while loop is used to extract each digit of the number one by one. The digit is obtained using the modulus operator `% 10`, and its cube is added to the sum variable using `digit ** 3`. The number is then reduced using `Math.floor(temp / 10)`.

After the loop completes, the program compares the calculated sum with the original number. If both values are equal, the number is an Armstrong number. Otherwise, it is not an Armstrong number.

Lab-10

- a) Write a JS conditional statement to find the sign of a product of three numbers using functions. Display an alert box with the specified sign.**

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>lab10a</title>

  <p>Write a JS conditional statement to find the sign of a product of three numbers using functions.

Display an alert box with the specified sign.</p>

</head>

<body>

  <script>

    function findProductSign(num1, num2, num3) {

      let product = num1 * num2 * num3;

      if (product > 0) {

        return "POSITIVE";

      } else if (product < 0) {

        return "NEGATIVE";

      } else {

        return "ZERO";

      }

    }

    let a = parseFloat(prompt("Enter first number:"));

    let b = parseFloat(prompt("Enter second number:"));

    let c = parseFloat(prompt("Enter third number:"));t
```

```

    if (!isNaN(a) && !isNaN(b) && !isNaN(c)) {
        let sign = findProductSign(a, b, c);
        let product = a * b * c;
        alert(`Numbers: ${a}, ${b}, ${c}\nProduct: ${product}\nSign: ${sign}`);
    } else {
        alert("Please enter valid numbers!");
    }
}
</script>
</body>
</html>

```

OUTPUT:



Explanation:

This HTML code checks the sign of the product of three numbers using JavaScript functions. The program first displays a short description of the task using a `<p>` element inside the webpage. The `findProductSign()` function is used to calculate the product of three numbers and determine whether the result is positive, negative, or zero. The function uses conditional statements (`if`, `else if`, and `else`) to return the appropriate sign. The program uses the `prompt()` function to take three numbers as input from the user. The entered values are converted into decimal numbers using `parseFloat()`. Before processing, the program checks whether the entered values are valid numbers using the `isNaN()` function. If any invalid input is detected, an error message is displayed using `alert()`. If all inputs are valid, the function is called to find the sign of the product. The product and its sign are then displayed in an alert box using template literals.

b) Write a JS conditional statement to sort three numbers using functions.

Display an alert box to show the result.

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>lab10b</title>

  <p>Write a JS conditional statement to sort three numbers using functions.
Display an alert box to show the result</p>

</head>

<body>

  <script>

    function sortThreeNumbers(num1, num2, num3) {

      let numbers = [num1, num2, num3];

      numbers.sort((a, b) => a - b);

      return numbers;

    }

    let a = parseFloat(prompt("Enter first number:"));

    let b = parseFloat(prompt("Enter second number:"));

    let c = parseFloat(prompt("Enter third number:"));

    if (!isNaN(a) && !isNaN(b) && !isNaN(c)) {

      let sortedNumbers = sortThreeNumbers(a, b, c);

      alert(`Numbers: ${a}, ${b}, ${c}\nSorted: ${sortedNumbers}`);

    } else {

      alert("Please enter valid numbers!");

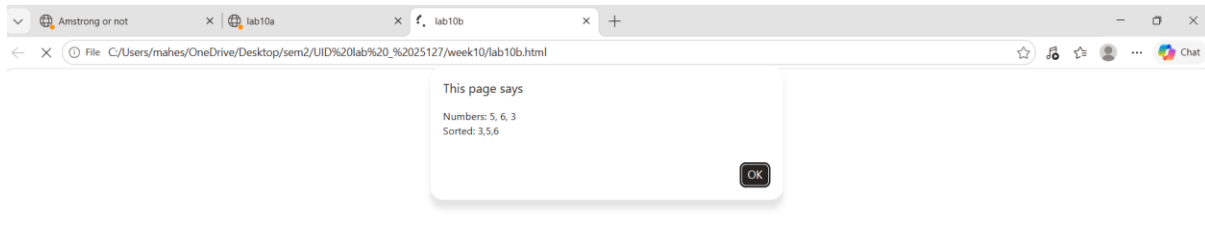
    }

  </script>
```

</body>

</html>

OUTPUT:



Explanation:

This HTML code sorts three numbers using JavaScript functions and displays the result in an alert box. The webpage first shows a short description of the task using a `<p>` element inside the `<head>` section. The `sortThreeNumbers()` function is used to sort the three entered numbers. The numbers are stored inside an array, and the `sort()` method is used with a comparison function `(a, b) => a - b` to arrange them in ascending order. The program uses the `prompt()` function to take three numbers as input from the user. The entered values are converted into decimal numbers using `parseFloat()`. Before sorting, the program checks whether all inputs are valid numbers using the `isNaN()` function. If invalid data is entered, an error message is displayed using `alert()`. If the inputs are valid, the sorting function is called, and the sorted numbers are stored in a variable. The original numbers and the sorted result are then displayed in an alert box using template literals.

c) Write a JS conditional statement to find the largest of five numbers using function & onclick event. Display an alert box to show the result.

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>lab10c</title>

    <p>Write a JS conditional statement to find the largest of five numbers using function &
onclick event.

Display an alert box to show the result</p>

</head>

<body>

    <script>

        function findLargest(num1, num2, num3, num4, num5) {

            let largest = num1;

            if (num2 > largest) {

                largest = num2;

            }

            if (num3 > largest) {

                largest = num3;

            }

            if (num4 > largest) {

                largest = num4;

            }

            if (num5 > largest) {

                largest = num5;

            }

        }

    </script>

</body>

</html>
```

```

        return largest;
    }

    function showLargest() {

        let a = parseFloat(prompt("Enter first number:"));
        let b = parseFloat(prompt("Enter second number:"));
        let c = parseFloat(prompt("Enter third number:"));
        let d = parseFloat(prompt("Enter fourth number:"));
        let e = parseFloat(prompt("Enter fifth number:"));

        if (!isNaN(a) && !isNaN(b) && !isNaN(c) && !isNaN(d) && !isNaN(e)) {

            let largest = findLargest(a, b, c, d, e);

            alert(`Numbers: ${a}, ${b}, ${c}, ${d}, ${e}\nLargest: ${largest}`);

        } else {

            alert("Please enter valid numbers!");

        }

    }

</script>

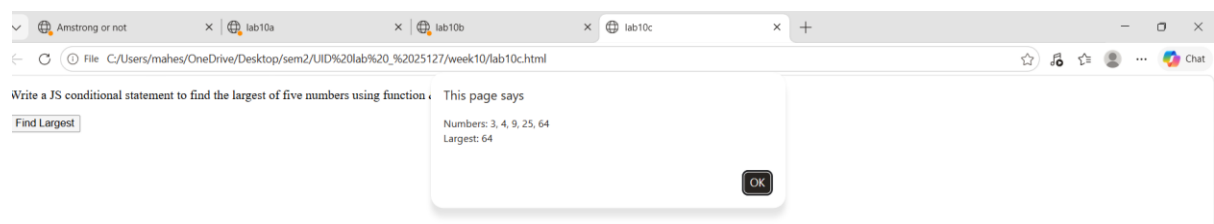
<button onclick="showLargest()">Find Largest</button>

</body>

</html>

```

OUTPUT:



Explanation:

This HTML code finds the largest among five numbers using JavaScript functions and an onclick event. The webpage contains a button labeled “Find Largest,” which triggers the program when clicked. The findLargest() function is used to compare five numbers and determine the largest value. The first number is initially stored in the variable largest, and conditional statements are used to compare it with the remaining numbers. If a bigger number is found, the value of largest is updated. The showLargest() function is executed when the button is clicked. It uses the prompt() function to take five numbers as input from the user and converts them into decimal numbers using parseFloat(). The program checks whether all entered values are valid numbers using the isNaN() function. If invalid input is entered, an error message is displayed using alert(). If all inputs are valid, the findLargest() function is called, and the largest number is stored in a variable. The entered numbers and the largest value are then displayed in an alert box using template literals.

d) Write a JS for loop that will iterate from 0 to 15. For each iteration, it will check if the current number is odd or even & display a message to the screen.

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>lab10d</title>

  <p>Write a JS for loop that will iterate from 0 to 15. For each iteration, it will check if the
current number is odd or even & display a message to the screen.

</p>

</head>

<body>

  <button onclick="startChecking()">Start</button>

  <div id="result"></div>

  <script>

    function startChecking() {

      let start = parseInt(prompt("Enter starting number:"));

      let end = parseInt(prompt("Enter ending number:"));

      if (isNaN(start) || isNaN(end)) {

        alert("Please enter valid numbers!");

        return;

      }

      let resultDiv = document.getElementById('result');

      resultDiv.innerHTML = `

        <h3>Range: ${start} to ${end}</h3>

        <button onclick="displayOdd(${start}, ${end})">Odd Numbers</button>

        <button onclick="displayEven(${start}, ${end})">Even Numbers</button>
```

```

        <div id="output" style="margin-top: 20px; padding: 10px; border: 1px solid
#ccc;"></div>

        `;
    }

    function displayOdd(start, end) {
        let oddNumbers = [];
        for (let i = start; i <= end; i++) {
            if (i % 2 !== 0) {
                oddNumbers.push(i);
            }
        }

        document.getElementById('output').innerHTML = `<strong>Odd Numbers:</strong>
${oddNumbers.join(", ")}`;
    }

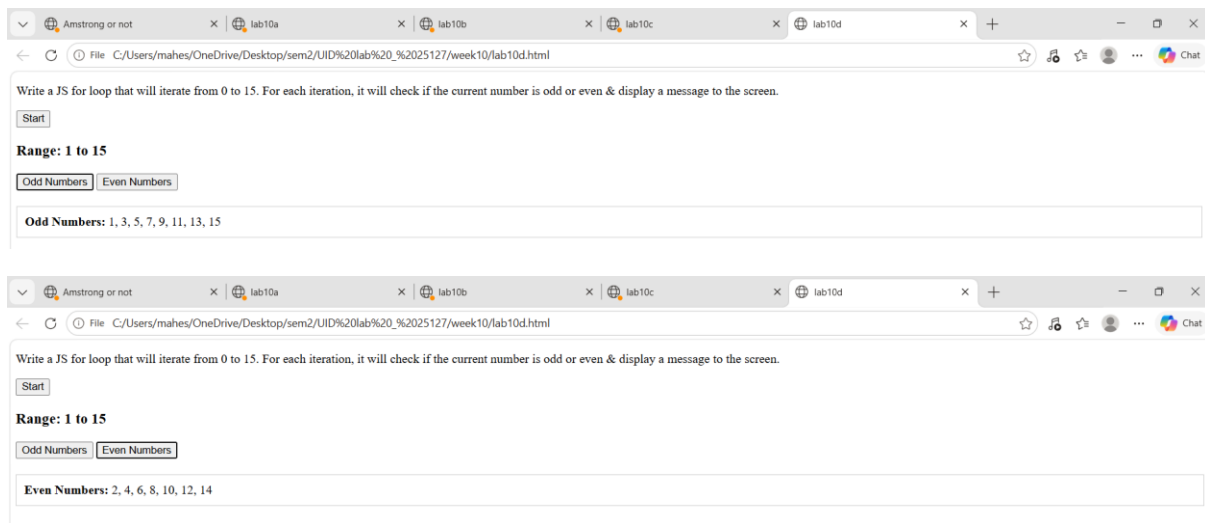
    function displayEven(start, end) {
        let evenNumbers = [];
        for (let i = start; i <= end; i++) {
            if (i % 2 === 0) {
                evenNumbers.push(i);
            }
        }

        document.getElementById('output').innerHTML = `<strong>Even
Numbers:</strong> ${evenNumbers.join(", ")}`;
    }

    </script>
</body>
</html>

```

OUTPUT:



Explanation:

This HTML code checks and displays odd and even numbers within a user-defined range using JavaScript loops and functions. The webpage contains a “Start” button, which begins the program when clicked. When the button is pressed, the `startChecking()` function is executed. It uses the `prompt()` function to take the starting and ending numbers from the user and converts them into integers using `parseInt()`. The program validates the inputs using the `isNaN()` function. If invalid values are entered, an error message is displayed using `alert()`. After successful input validation, the program dynamically creates two buttons: “Odd Numbers” and “Even Numbers.” It also creates an output section using `innerHTML` to display the results. The `displayOdd()` function uses a for loop to iterate through the range and checks whether each number is odd using the condition `i % 2 !== 0`. All odd numbers are stored in an array and displayed on the webpage. Similarly, the `displayEven()` function checks for even numbers using the condition `i % 2 === 0`. The even numbers are stored in an array and displayed in the output section.

e) Write a JS program which iterates the integers from 1 to 100.

But for multiples of three print "LOL" instead of the number and for multiples of five print "OK".

For numbers which are multiples of both three & five print "LOL-OK".

CODE:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>lab10e</title>
```

```
  <p>. Write a JS program which iterates the integers from 1 to 100.
```

```
But for multiples of three print "LOL" instead of the number and for multiples of five print "OK".
```

```
For numbers which are multiples of both three & five print "LOL-OK".
```

```
</p>
```

```
</head>
```

```
<body>
```

```
  <h2>LOL-OK Game</h2>
```

```
  <button onclick="startChecking()">Start</button>
```

```
  <div id="output" style="margin-top: 20px; padding: 10px; border: 1px solid #ccc; min-height: 50px;"></div>
```

```
<script>
```

```
  function startChecking() {
```

```
    let start = parseInt(prompt("Enter the starting number:"));
```

```
    let end = parseInt(prompt("Enter the ending number:"));
```

```
    if (isNaN(start) || isNaN(end) || start > end) {
```

```

        alert("Please enter valid numbers with start less than or equal to end.");

        return;
    }

    let result = [];

    for (let i = start; i <= end; i++) {
        if (i % 3 === 0 && i % 5 === 0) {
            result.push("LOL-OK");
        } else if (i % 3 === 0) {
            result.push("LOL");
        } else if (i % 5 === 0) {
            result.push("OK");
        } else {
            result.push(i);
        }
    }

    document.getElementById('output').innerHTML = `<strong>Output (${start} to
    ${end}):</strong><br>${result.join(", ")}`;
}

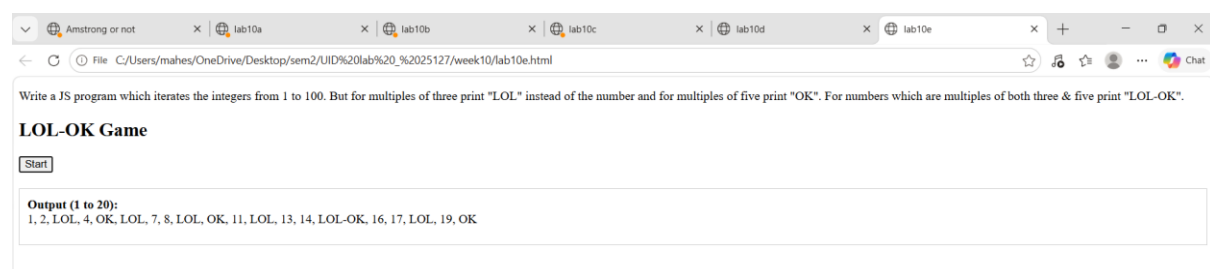
</script>

</body>

</html>

```

OUTPUT:



Explanation:

This HTML code implements a simple LOL-OK game using JavaScript loops and conditional statements. The webpage contains a “Start” button that begins the program when clicked. When the button is pressed, the startChecking() function is executed. It takes the starting and ending numbers as input from the user using the prompt() function and converts them into integers using parseInt(). The program validates the input values using the isNaN() function and also checks whether the starting number is less than or equal to the ending number. If invalid input is entered, an error message is displayed using alert(). A for loop is used to iterate through all numbers within the given range. Conditional statements check whether the current number is divisible by 3, 5, or both using the modulus operator %. If a number is divisible by both 3 and 5, the program stores "LOL-OK" in the result array. If it is divisible only by 3, it stores "LOL", and if divisible only by 5, it stores "OK". Otherwise, the actual number is stored.

Lab-11

1.A website requires users to enter a username. The system should display a message while typing if the username is less than 5 characters.

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Username Validation</title>

  <style>

    body {

      font-family: Arial, sans-serif;

      max-width: 500px;

      margin: 50px auto;

      padding: 20px;

    }

    .form-group {

      margin-bottom: 20px;

    }

    label {

      display: block;

      margin-bottom: 8px;

      font-weight: bold;

    }

    input {

      width: 100%;

      padding: 10px;

      font-size: 16px;
```

```
        border: 2px solid #ccc;
        border-radius: 4px;
        box-sizing: border-box;
    }
    input:focus {
        outline: none;
        border-color: #4CAF50;
    }
    .message {
        margin-top: 10px;
        padding: 10px;
        border-radius: 4px;
        display: none;
        font-weight: bold;
    }
    .warning {
        color: #d32f2f;
        background-color: #ffebee;
        display: block;
    }
    .success {
        color: #388e3c;
        background-color: #e8f5e9;
        display: block;
    }
</style>
</head>
<body>
    <h1>Username Validation</h1>
```

```

<form>

  <div class="form-group">

    <label for="username">Enter Username:</label>

    <input type="text" id="username" placeholder="Minimum 5 characters required">

    <div id="message" class="message"></div>

  </div>

</form>


<script>

  const usernameInput = document.getElementById('username');
  const messageDiv = document.getElementById('message');

  usernameInput.addEventListener('input', function() {

    const username = this.value;

    if (username.length < 5 && username.length > 0) {

      messageDiv.textContent = `⚠ Username must be at least 5 characters
(${username.length}/5)`;

      messageDiv.className = 'message warning';

    } else if (username.length >= 5) {

      messageDiv.textContent = `✓ Username is valid (${username.length} characters)`;
      messageDiv.className = 'message success';

    } else {

      messageDiv.className = 'message';

    }

  });

</script>

</body>

</html>

```

OUTPUT:

Username Validation

Enter Username:

mahe@123

✓ Username is valid (8 characters)

Explanation:

This HTML code creates a Username Validation form using HTML, CSS, and JavaScript. The webpage contains a text input field where the user can enter a username and receive validation messages instantly. The CSS section is used to style the webpage, input field, and message box. Different styles are applied for warning and success messages using the `.warning` and `.success` classes. The JavaScript code first selects the username input field and message display area using `getElementById()`. An input event listener is attached to the text field so that validation occurs whenever the user types. When the user enters text, the program checks the length of the username using conditional statements. If the username contains fewer than 5 characters, a warning message is displayed in red color. If the username length is 5 or more characters, a success message is displayed in green color indicating that the username is valid. If the input field is empty, the message box is hidden. The `textContent` property is used to display messages dynamically, and `className` is used to apply the appropriate CSS styling. Overall, the code demonstrates form validation, event handling, DOM manipulation, and dynamic styling using JavaScript.

2. A registration form should validate email format before submission. If invalid, the form should not be submitted.

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Registration Form with Email Validation</title>

  <style>

    body {

      font-family: Arial, sans-serif;

      max-width: 500px;

      margin: 50px auto;

      padding: 20px;

      background-color: #f5f5f5;

    }

    .form-container {

      background-color: white;

      padding: 30px;

      border-radius: 8px;

      box-shadow: 0 2px 10px rgba(0,0,0,0.1);

    }

    h1 {

      color: #333;

      text-align: center;

    }

    .form-group {

      margin-bottom: 20px;
```

```
}  
label {  
    display: block;  
    margin-bottom: 8px;  
    font-weight: bold;  
    color: #333;  
}  
input[type="text"],  
input[type="email"],  
input[type="password"] {  
    width: 100%;  
    padding: 10px;  
    font-size: 16px;  
    border: 2px solid #ccc;  
    border-radius: 4px;  
    box-sizing: border-box;  
}  
input:focus {  
    outline: none;  
    border-color: #4CAF50;  
    background-color: #f0f8f0;  
}  
input[type="submit"] {  
    width: 100%;  
    padding: 12px;  
    font-size: 16px;  
    background-color: #4CAF50;  
    color: white;  
    border: none;
```

```
border-radius: 4px;
cursor: pointer;
font-weight: bold;
transition: background-color 0.3s;
}
input[type="submit"]:hover {
background-color: #45a049;
}
input[type="submit"]:disabled {
background-color: #cccccc;
cursor: not-allowed;
}
.error-message {
color: #d32f2f;
background-color: #ffebee;
padding: 10px;
border-radius: 4px;
margin-top: 5px;
font-size: 14px;
display: none;
}
.success-message {
color: #388e3c;
background-color: #e8f5e9;
padding: 10px;
border-radius: 4px;
margin-top: 5px;
font-size: 14px;
display: none;
```

```

    }

</style>

</head>

<body>

  <div class="form-container">

    <h1>Registration Form</h1>

    <form id="registrationForm" onsubmit="return validateForm(event)">

      <div class="form-group">

        <label for="fullname">Full Name:</label>

        <input type="text" id="fullname" name="fullname" required>

      </div>

      <div class="form-group">

        <label for="email">Email Address:</label>

        <input type="email" id="email" name="email" required>

        <div id="emailError" class="error-message"></div>

        <div id="emailSuccess" class="success-message"></div>

      </div>

      <div>

        <input type="submit" value="SUBMIT">

        <onclick="alert('Form submitted successfully!')">

      </div>

    </form>

  </div>

  <script>

    function validateEmail(email) {

      const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

      return emailRegex.test(email);

    }

  </script>

```

```

    }

function validateForm(event) {
    const email = document.getElementById("email").value.trim();
    const emailError = document.getElementById("emailError");
    const emailSuccess = document.getElementById("emailSuccess");
    const submitButton = document.querySelector('input[type="submit"]');

    emailError.style.display = "none";
    emailSuccess.style.display = "none";
    emailError.textContent = "";
    emailSuccess.textContent = "";

    if (!validateEmail(email)) {
        event.preventDefault();

        emailError.textContent = "Please enter a valid email address (e.g.,
user@example.com)";
        emailError.style.display = "block";
        submitButton.disabled = true;
        return false;
    }

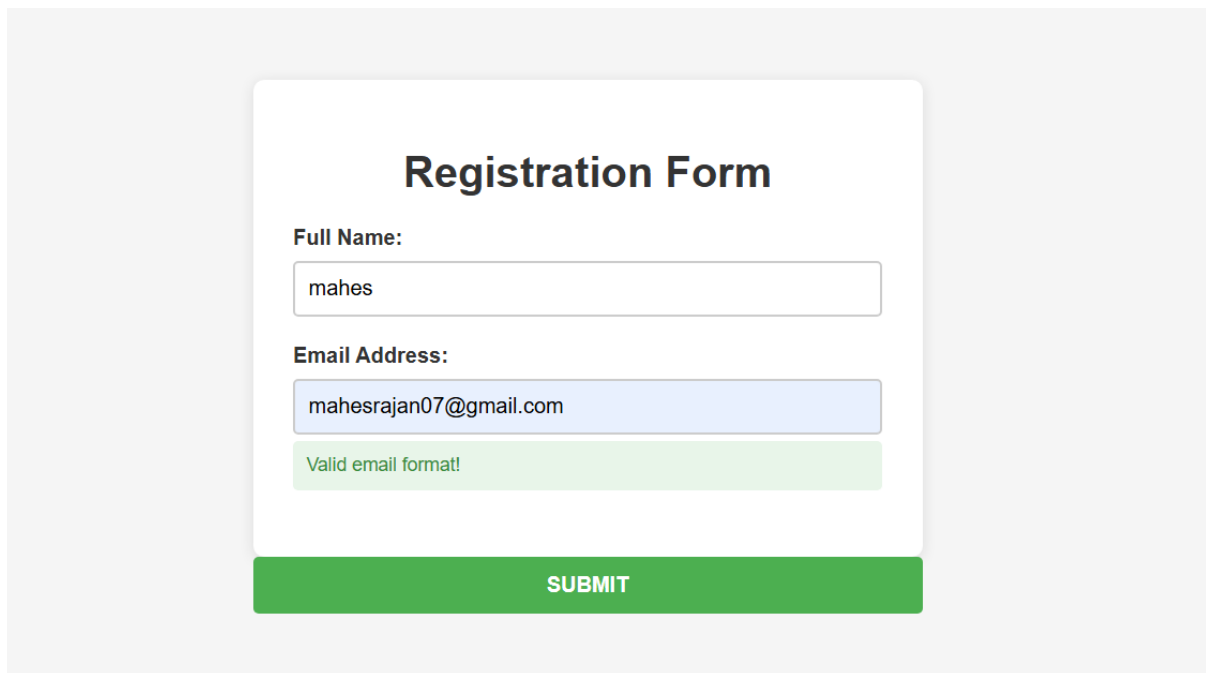
    emailSuccess.textContent = "Email format is valid!";
    emailSuccess.style.display = "block";
    submitButton.disabled = false;

    return true;
}

```

```
document.getElementById("email").addEventListener("input", function() {  
    const email = this.value.trim();  
    const emailError = document.getElementById("emailError");  
    const emailSuccess = document.getElementById("emailSuccess");  
  
    if (email === "") {  
        emailError.style.display = "none";  
        emailSuccess.style.display = "none";  
        return;  
    }  
  
    if (validateEmail(email)) {  
        emailError.style.display = "none";  
        emailSuccess.textContent = "Valid email format!";  
        emailSuccess.style.display = "block";  
    } else {  
        emailSuccess.style.display = "none";  
        emailError.textContent = "Invalid email format. Use: user@example.com";  
        emailError.style.display = "block";  
    }  
});  
</script>  
</body>  
</html>
```

OUTPUT:



The image shows a web form titled "Registration Form". It contains two input fields: "Full Name" with the value "mahes" and "Email Address" with the value "mahesrajan07@gmail.com". Below the email field is a green message box that says "Valid email format!". At the bottom of the form is a green button labeled "SUBMIT".

Explanation:

This HTML code creates a Registration Form with Email Validation using HTML, CSS, and JavaScript. The webpage contains input fields for full name and email address along with a submit button. The CSS section is used to style the form, input fields, buttons, and validation messages. Different styles are applied for error messages and success messages using the `.error-message` and `.success-message` classes. The JavaScript code contains a `validateEmail()` function that uses a regular expression to check whether the entered email format is valid. It returns true for valid email addresses and false for invalid ones. The `validateForm()` function is executed when the form is submitted. It checks the entered email address and displays an error message if the format is invalid. If the email is valid, a success message is displayed and the form submission is allowed. An input event listener is attached to the email field so validation happens instantly while the user types. The program dynamically shows either a success message or an error message based on the entered email format. The `textContent`, `style.display`, and `disabled` properties are used to update messages, control visibility, and enable or disable the submit button dynamically. Overall, the code demonstrates form validation, regular expressions, event handling, DOM manipulation, and dynamic styling using JavaScript.

3. A system should check password strength while typing and display whether it is weak or strong.

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>3. A system should check password strength while typing and display whether it is weak or strong.

</title>

</head>

<body>

  <h1>Password Strength Checker</h1>

  <style>

    body {

      font-family: Arial, sans-serif;

      max-width: 400px;

      margin: 50px auto;

      padding: 20px;

    }

    input[type="password"] {

      width: 100%;

      padding: 10px;

      font-size: 16px;

      border: 2px solid #ccc;

      border-radius: 4px;

      box-sizing: border-box;

    }

  </style>

</body>

</html>
```

```
#strengthMessage {  
    margin-top: 10px;  
    font-weight: bold;  
}  
</style>
```

```
<p>Enter a password to check its strength:</p>  
<input type="password" id="password" placeholder="Enter your password">  
<div id="strengthMessage"></div>
```

```
<script>  
    const passwordInput = document.getElementById('password');  
    const strengthMessage = document.getElementById('strengthMessage');  
  
    passwordInput.addEventListener('input', function() {  
        const password = passwordInput.value;  
        let strength = 'Weak';  
        let color = 'red';  
  
        if (password.length >= 8 && /[A-Z]/.test(password) && /[0-9]/.test(password)) {  
            strength = 'Medium';  
            color = 'orange';  
        }  
  
        if (password.length >= 8 && /[A-Z]/.test(password) && /[0-9]/.test(password) &&  
            /[!@#$%^&*]/.test(password)) {  
            strength = 'Strong';  
            color = 'green';  
        }  
    })  
</script>
```

```

        strengthMessage.textContent = `Password Strength: ${strength}`;

        strengthMessage.style.color = color;

    });

</script>

</body>

</html>

```

OUTPUT:

Password Strength Checker

Enter a password to check its strength:

Password Strength: Strong

Explanation:

This HTML code creates a Password Strength Checker using HTML, CSS, and JavaScript. The webpage contains a password input field where the user can type a password and instantly check its strength.

The CSS section is used to style the webpage, password input field, and strength message display area. Different colors are used to represent weak, medium, and strong passwords.

The JavaScript code first selects the password input field and the message display element using `getElementById()`. An input event listener is attached to the password field so that validation occurs while the user types.

Initially, the password strength is set to “Weak” with red color. The program then checks different conditions such as password length, uppercase letters, numbers, and special characters using regular expressions.

If the password contains at least 8 characters, one uppercase letter, and one number, the strength changes to “Medium” with orange color. If it also contains a special character, the strength becomes “Strong” with green color.

The `textContent` property is used to display the password strength message dynamically, and `style.color` changes the text color based on the password strength. Overall, the code demonstrates real-time validation, event handling, regular expressions, DOM manipulation, and dynamic styling using JavaScript.

4. A form should validate a mobile number when the user leaves the input field. The number must be exactly 10 digits.

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>A form should validate a mobile number when the user leaves the input field. The
number must be exactly 10 digits.</title>

</head>

<body>

  <style>

    body {

      font-family: Arial, sans-serif;

      max-width: 400px;

      margin: 50px auto;

      padding: 20px;

    }

    input[type="text"] {

      width: 100%;

      padding: 10px;

      font-size: 16px;

      border: 2px solid #ccc;

      border-radius: 4px;

      box-sizing: border-box;

    }

    input:focus {

      outline: none;
```

```

        border-color: #4CAF50;
    }

    #message {
        margin-top: 10px;
        padding: 10px;
        border-radius: 4px;
        display: none;
        font-weight: bold;
    }

    .error {
        color: #d32f2f;
        background-color: #ffebee;
        display: block;
    }

    .success {
        color: #388e3c;
        background-color: #e8f5e9;
        display: block;
    }
}
</style>

<h1>Mobile Number Validation</h1>

<form id="mobileForm">
    <label for="mobile">Enter Mobile Number:</label>
    <input type="text" id="mobile" placeholder="Enter 10-digit mobile number">
    <div id="message"></div>
    <button type="submit">Submit</button>
</form>

<script>
    const mobileInput = document.getElementById('mobile');

```

```
const messageDiv = document.getElementById('message');
const mobileForm = document.getElementById('mobileForm');

mobileInput.addEventListener('blur', function() {
  const mobileNumber = mobileInput.value.trim();
  const mobileRegex = /^\\d{10}$/; // Regular expression for exactly 10 digits

  if (mobileRegex.test(mobileNumber)) {
    messageDiv.textContent = "Valid mobile number!";
    messageDiv.className = "success";
  } else {
    messageDiv.textContent = "Invalid mobile number. Please enter exactly 10 digits.";
    messageDiv.className = "error";
  }
});

mobileForm.addEventListener('submit', function(event) {
  event.preventDefault(); // Prevent form submission
  const mobileNumber = mobileInput.value.trim();
  const mobileRegex = /^\\d{10}$/; // Regular expression for exactly 10 digits

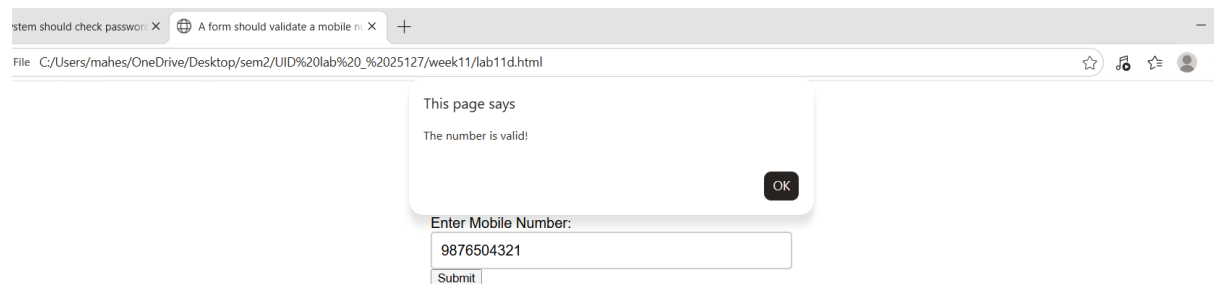
  if (mobileRegex.test(mobileNumber)) {
    alert("The number is valid!");
    alert("Form submitted successfully!");
  } else {
    alert("Invalid mobile number. Please enter exactly 10 digits.");
  }
});
```

```
</script>

</body>

</html>
```

OUTPUT:



Explanation:

This HTML code creates a Mobile Number Validation form using HTML, CSS, and JavaScript. The webpage contains a text input field where the user can enter a mobile number and validate it.

The CSS section is used to style the webpage, input field, buttons, and validation messages. Different styles are applied for valid and invalid messages using the `.success` and `.error` classes.

The JavaScript code first selects the mobile input field, message display area, and form element using `getElementById()`. Event listeners are used to handle validation when the user leaves the input field and when the form is submitted.

The blur event listener checks the mobile number when the user leaves the input field. A regular expression `/^\d{10}$/` is used to verify whether the entered number contains exactly 10 digits.

If the number is valid, a success message is displayed in green color. Otherwise, an error message is displayed in red color informing the user to enter exactly 10 digits.

The submit event listener prevents the default form submission using `event.preventDefault()`. It again validates the mobile number and displays alert messages based on whether the entered number is valid or invalid.

5. Design an HTML page, such that the registration form must validate:

Name (not empty)

Email (valid format)

Password (minimum 6 characters)

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Registration Form with Validation</title>

  <style>

    * {

      margin: 0;

      padding: 0;

      box-sizing: border-box;

    }

    body {

      font-family: Arial, sans-serif;

      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);

      min-height: 100vh;

      display: flex;

      justify-content: center;

      align-items: center;

      padding: 20px;

    }

  </style>

</head>

<body>

  <div>

    <h1>Registration</h1>

    <div>

      <div>Name</div>

      <div>Email</div>

      <div>Password</div>

    </div>

    <div>

      <div>Name</div>

      <div>Email</div>

      <div>Password</div>

    </div>

  </div>

</body>

</html>
```

```
.container {  
    background: white;  
    padding: 40px;  
    border-radius: 8px;  
    box-shadow: 0 10px 25px rgba(0, 0, 0, 0.2);  
    width: 100%;  
    max-width: 400px;  
}
```

```
h1 {  
    text-align: center;  
    color: #333;  
    margin-bottom: 30px;  
    font-size: 28px;  
}
```

```
.form-group {  
    margin-bottom: 20px;  
}
```

```
label {  
    display: block;  
    margin-bottom: 8px;  
    color: #333;  
    font-weight: bold;  
    font-size: 14px;  
}
```

```
input[type="text"],
```

```
input[type="email"],
input[type="password"] {
    width: 100%;
    padding: 12px;
    border: 2px solid #ddd;
    border-radius: 4px;
    font-size: 14px;
    transition: border-color 0.3s ease;
}
```

```
input[type="text"]:focus,
input[type="email"]:focus,
input[type="password"]:focus {
    outline: none;
    border-color: #667eea;
}
```

```
input.error {
    border-color: #e74c3c;
    background-color: #fadbd8;
}
```

```
input.success {
    border-color: #27ae60;
    background-color: #d5f4e6;
}
```

```
.error-message {
    color: #e74c3c;
}
```

```
font-size: 12px;  
margin-top: 5px;  
display: none;  
}
```

```
.error-message.show {  
    display: block;  
}
```

```
.success-message {  
    color: #27ae60;  
    font-size: 12px;  
    margin-top: 5px;  
    display: none;  
}
```

```
.success-message.show {  
    display: block;  
}
```

```
button {  
    width: 100%;  
    padding: 12px;  
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);  
    color: white;  
    border: none;  
    border-radius: 4px;  
    font-size: 16px;  
    font-weight: bold;
```

```
    cursor: pointer;

    transition: transform 0.2s ease, box-shadow 0.2s ease;

    margin-top: 10px;
}

button:hover {

    transform: translateY(-2px);

    box-shadow: 0 5px 15px rgba(102, 126, 234, 0.4);
}

button:active {

    transform: translateY(0);
}

.form-message {

    padding: 12px;

    border-radius: 4px;

    margin-bottom: 20px;

    display: none;

    text-align: center;

    font-weight: bold;
}

.form-message.success {

    background-color: #d5f4e6;

    color: #27ae60;

    display: block;
}
```

```
.form-message.error {  
    background-color: #fadbd8;  
    color: #e74c3c;  
    display: block;  
}
```

```
.validation-requirements {  
    background-color: #ecf0f1;  
    padding: 15px;  
    border-radius: 4px;  
    margin-bottom: 20px;  
    font-size: 13px;  
    color: #555;  
}
```

```
.validation-requirements h3 {  
    margin-bottom: 10px;  
    color: #333;  
    font-size: 14px;  
}
```

```
.validation-requirements ul {  
    list-style: none;  
    padding-left: 0;  
}
```

```
.validation-requirements li {  
    padding: 5px 0;  
}
```

```

.validation-requirements li:before {
    content: "• ";
    color: #667eea;
    font-weight: bold;
    margin-right: 8px;
}
</style>
</head>
<body>
<div class="container">
    <h1>Registration Form</h1>

    <div class="validation-requirements">
        <h3>Validation Requirements:</h3>
        <ul>
            <li>Name: Not empty</li>
            <li>Email: Valid format</li>
            <li>Password: Minimum 6 characters</li>
        </ul>
    </div>

    <div id="formMessage" class="form-message"></div>

    <form id="registrationForm" onsubmit="submitForm(event)">
        <!-- Name Field -->
        <div class="form-group">
            <label for="name">Full Name:</label>
            <input

```

```
        type="text"
        id="name"
        name="name"
        placeholder="Enter your full name"
        onblur="validateName()"
    >

    <div id="nameError" class="error-message"></div>

    <div id="nameSuccess" class="success-message">✓ Name is valid</div>
</div>
```

```
<!-- Email Field -->
<div class="form-group">
    <label for="email">Email Address:</label>
    <input
        type="email"
        id="email"
        name="email"
        placeholder="Enter your email address"
        onblur="validateEmail()"
    >

    <div id="emailError" class="error-message"></div>

    <div id="emailSuccess" class="success-message">✓ Email is valid</div>
</div>
```

```
<!-- Password Field -->
<div class="form-group">
    <label for="password">Password:</label>
    <input
        type="password"
```

```

        id="password"
        name="password"
        placeholder="Minimum 6 characters"
        onblur="validatePassword()"
    >

    <div id="passwordError" class="error-message"></div>

    <div id="passwordSuccess" class="success-message">✓ Password is valid</div>

</div>

<button type="submit">Register</button>

</form>

</div>

<script>
    // Validate Name
    function validateName() {
        const nameInput = document.getElementById('name');
        const nameError = document.getElementById('nameError');
        const nameSuccess = document.getElementById('nameSuccess');
        const name = nameInput.value.trim();

        if (name === "") {
            nameInput.classList.remove('success');
            nameInput.classList.add('error');
            nameError.textContent = 'X Name cannot be empty';
            nameError.classList.add('show');
            nameSuccess.classList.remove('show');
            return false;
        } else if (name.length < 2) {

```

```

        nameInput.classList.remove('success');
        nameInput.classList.add('error');
        nameError.textContent = 'X Name must be at least 2 characters';
        nameError.classList.add('show');
        nameSuccess.classList.remove('show');
        return false;
    } else {
        nameInput.classList.remove('error');
        nameInput.classList.add('success');
        nameError.classList.remove('show');
        nameSuccess.classList.add('show');
        return true;
    }
}

```

// Validate Email

```

function validateEmail() {
    const emailInput = document.getElementById('email');
    const emailError = document.getElementById('emailError');
    const emailSuccess = document.getElementById('emailSuccess');
    const email = emailInput.value.trim();

    // Regular expression for email validation
    const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

    if (email === "") {
        emailInput.classList.remove('success');
        emailInput.classList.add('error');
        emailError.textContent = 'X Email cannot be empty';
    }
}

```

```

        emailError.classList.add('show');
        emailSuccess.classList.remove('show');
        return false;
    } else if (!emailPattern.test(email)) {
        emailInput.classList.remove('success');
        emailInput.classList.add('error');
        emailError.textContent = 'X Please enter a valid email format (e.g.,
user@example.com)';
        emailError.classList.add('show');
        emailSuccess.classList.remove('show');
        return false;
    } else {
        emailInput.classList.remove('error');
        emailInput.classList.add('success');
        emailError.classList.remove('show');
        emailSuccess.classList.add('show');
        return true;
    }
}

```

// Validate Password

```

function validatePassword() {
    const passwordInput = document.getElementById('password');
    const passwordError = document.getElementById('passwordError');
    const passwordSuccess = document.getElementById('passwordSuccess');
    const password = passwordInput.value;

    if (password === "") {
        passwordInput.classList.remove('success');
    }
}

```

```

    passwordInput.classList.add('error');
    passwordError.textContent = 'X Password cannot be empty';
    passwordError.classList.add('show');
    passwordSuccess.classList.remove('show');
    return false;
} else if (password.length < 6) {
    passwordInput.classList.remove('success');
    passwordInput.classList.add('error');
    passwordError.textContent = `X Password must be at least 6 characters
($ {password.length}/6)`;
    passwordError.classList.add('show');
    passwordSuccess.classList.remove('show');
    return false;
} else {
    passwordInput.classList.remove('error');
    passwordInput.classList.add('success');
    passwordError.classList.remove('show');
    passwordSuccess.classList.add('show');
    return true;
}
}

```

// Submit Form

```

function submitForm(event) {
    event.preventDefault();

    // Validate all fields
    const isNameValid = validateName();
    const isEmailValid = validateEmail();

```

```

const isPasswordValid = validatePassword();

const formMessage = document.getElementById('formMessage');

if (isNameValid && isEmailValid && isPasswordValid) {
    formMessage.textContent = '✓ Registration successful! All fields are valid.';
    formMessage.classList.remove('error');
    formMessage.classList.add('success');

    // Reset form after 2 seconds
    setTimeout(() => {
        document.getElementById('registrationForm').reset();
        document.getElementById('name').classList.remove('success');
        document.getElementById('email').classList.remove('success');
        document.getElementById('password').classList.remove('success');
        document.getElementById('nameSuccess').classList.remove('show');
        document.getElementById('emailSuccess').classList.remove('show');
        document.getElementById('passwordSuccess').classList.remove('show');
        formMessage.classList.remove('success');
    }, 2000);
} else {
    formMessage.textContent = 'X Please correct the errors above and try again.';
    formMessage.classList.remove('success');
    formMessage.classList.add('error');
}
}

// Clear form message on input
document.getElementById('name').addEventListener('input', function() {

```

```

        document.getElementById('formMessage').classList.remove('success', 'error');
    });

    document.getElementById('email').addEventListener('input', function() {
        document.getElementById('formMessage').classList.remove('success', 'error');
    });

    document.getElementById('password').addEventListener('input', function() {
        document.getElementById('formMessage').classList.remove('success', 'error');
    });
</script>
</body>
</html>

```

OUTPUT:

Registration Form

Validation Requirements:

- Name: Not empty
- Email: Valid format
- Password: Minimum 6 characters

Full Name:

✓ Name is valid

Email Address:

✓ Email is valid

Password:

✓ Password is valid

Register

Explanation:

This HTML code creates a Registration Form with Validation using HTML, CSS, and JavaScript. The form contains input fields for name, email, and password, along with validation rules for each field.

The CSS section is used to style the webpage, form container, input fields, buttons, and validation messages. Different colors and styles are applied for valid and invalid inputs using `.success` and `.error` classes.

The JavaScript code contains separate functions `validateName()`, `validateEmail()`, and `validatePassword()` to validate each input field individually. These functions check conditions such as empty values, valid email format, and minimum password length.

The program uses regular expressions to validate the email format and conditional statements to display appropriate error or success messages dynamically. The `classList` property is used to add or remove CSS classes based on validation results.

When the form is submitted, the `submitForm()` function prevents the default form submission using `event.preventDefault()`. It validates all fields and displays a success message if all inputs are correct; otherwise, it shows an error message.

Event listeners are also used to clear form messages while the user types in the input fields. Overall, the code demonstrates form validation, event handling, regular expressions, DOM manipulation, dynamic styling, and user input handling using JavaScript.

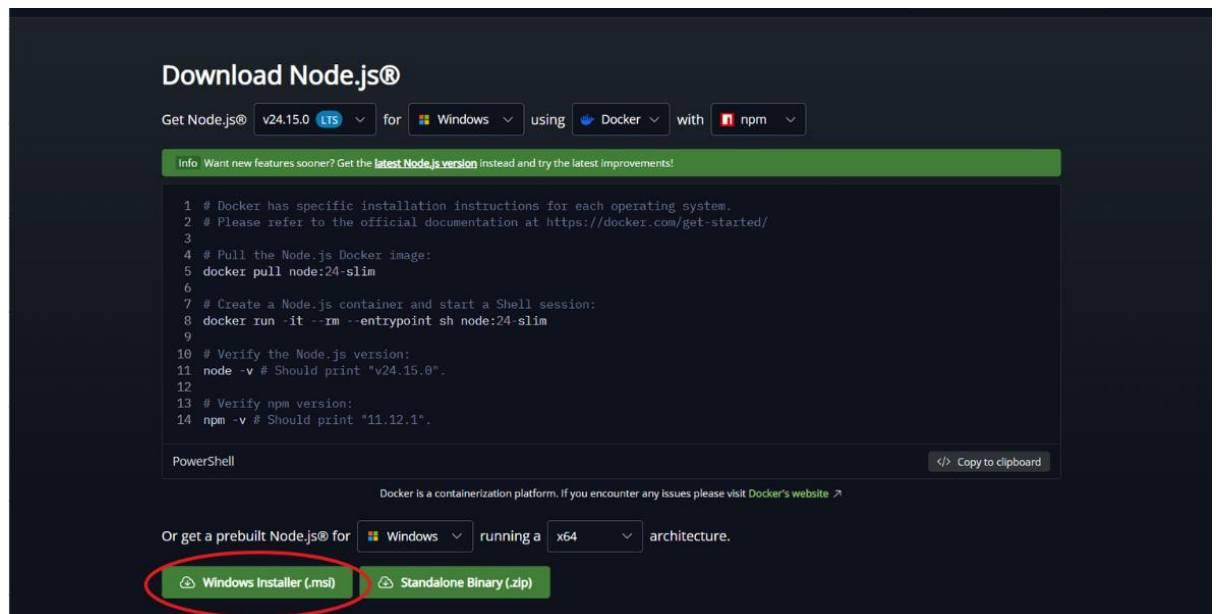
LAB-12

(A) To understand the installation process of a electron.

Steps to Install and Set Up Electron

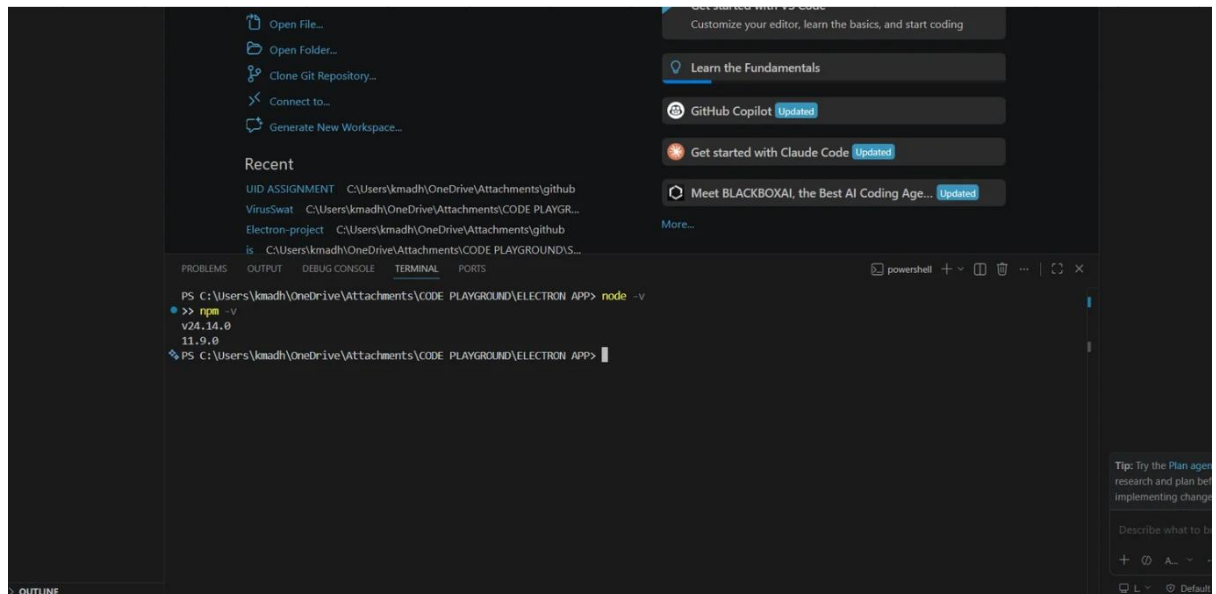
1. Install Prerequisite: Node.js

The first step is to install Node.js from the official Node.js website. Installing Node.js automatically installs NPM (Node Package Manager), which is required to manage Electron packages and dependencies.



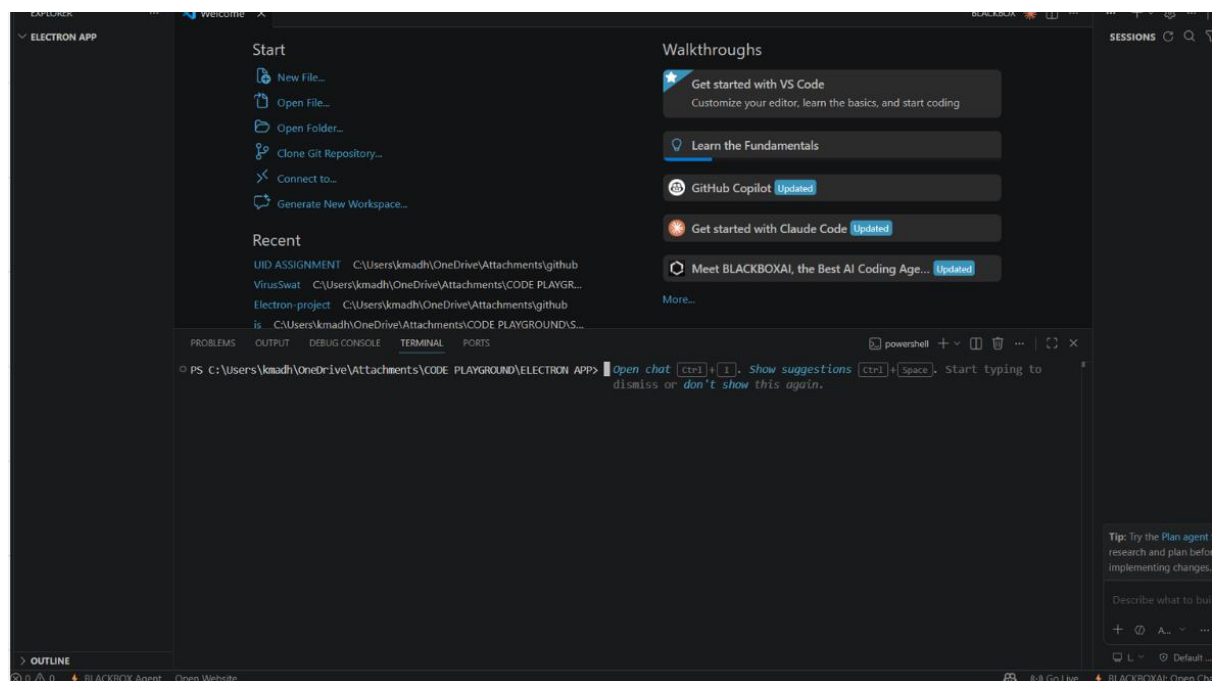
2. Verify Installation

After installation, open the command prompt or terminal and check the installed versions of Node.js and NPM using version commands. If the version numbers are displayed successfully, the installation process is completed correctly.



3. Create Project Folder

Create a new folder named “electron-app” and open it using Visual Studio Code. This folder will contain all the project files related to the Electron application.



4. Initialize the Project

Initialize the project using NPM. This process creates a package.json file, which stores project information, scripts, and dependency details required for the Electron application.

```
</> Bash  
npm init -y
```

5. Install Electron

Install Electron inside the project using NPM. Electron is downloaded into the project folder and saved as a development dependency inside the package.json file.

```
</> Bash  
npm install electron --save-dev
```

6. Project Structure After Installation

After installation, the project folder contains files such as package.json, node_modules, and other required project files. These files manage dependencies and application configuration.

```
my-electron-app/  
├── node_modules/ ← Electron installed here  
├── package.json  
└── package-lock.json
```

7. Add Start Script

Open the package.json file and add a start script. The start script is used to launch the Electron application directly using an NPM command.

```
</> JSON  
{  
  "scripts": {  
    "start": "electron ."  
  }  
}
```

8. Run the Application

Finally, run the Electron application using the appropriate NPM command in the terminal. If the setup is correct, the Electron desktop application window will open successfully.

A screenshot of a terminal window with a dark background. The prompt is a white icon of a code editor followed by the word "Bash" in white. The command "npm start" is entered in orange text.

```
</> Bash  
npm start
```